

iSMAGi®

Institut Supérieur de Management
d'Administration et de Génie Informatique
المعهد العالي للتدبير و الإدارة والهندسة المعلوماتية

Reinforcement Learning

Presented by :
Pr. Khalid HADDOUCH
Professor at l'ENSA of Fez
SmartMedina Project Coordinator
ISA Laboratory

Contact :

Khalid.haddouch@usmba.ac.ma

https://www.linkedin.com/in/khalid-haddouch

Phone : +212 6 61 42 79 30

Plan

- Chapter 1 : General introduction
- Chapter 2 : classical algorithms for reinforcement Learning
- Chapter 3 : Advanced Reinforcement Learning
- Chapter 4 : Real applications

Chaper1 : General introduction

Goals

- Understand the concept of Reinforcement Learning (RL).
- Distinguish between RL, supervised and unsupervised learning.
- Identify the fundamental elements: agent, environment, reward.
- Explore some concrete applications in engineering.
- Discover the main Python libraries and tools used in Reinforcement Learning.

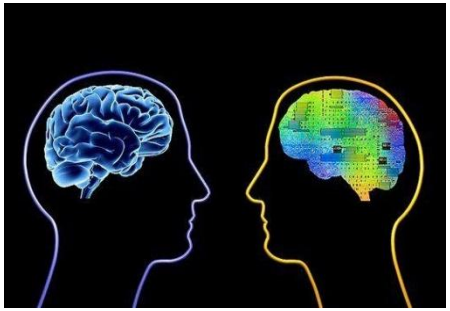
Outlines

- Introduction
- Different subfields of Machine Learning
- Training Machine Learning Models
- Materials and tools for Reinforcement Learning
- Applications and Engineering Domains
- Conclusion

Introduction

Introduction

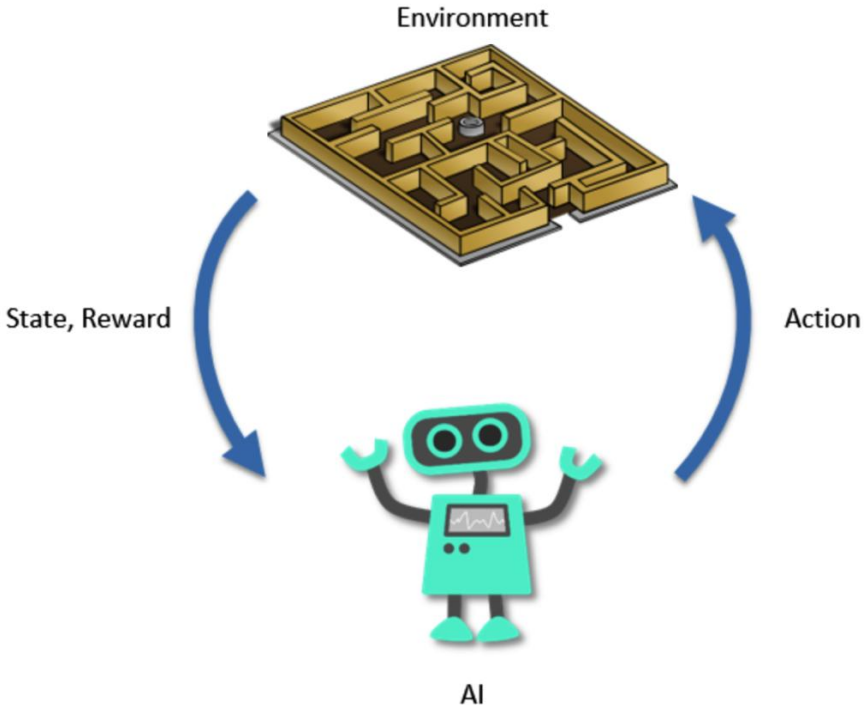
- **Machine Learning**
  - is a subset of artificial intelligence (AI).
  - that enables computers to learn and make decisions from data without being explicitly programmed for every specific task.
  - Machine learning techniques find patterns in data to create a model that can predict or make decisions on new invisible data.
  - In recent years, it has become very popular in various fields of application.



https://www.kdnuggets.com/2019/09/beyond-neurons-five-cognitive-functions-human-brain-recreate-artificial-intelligence.html

Introduction

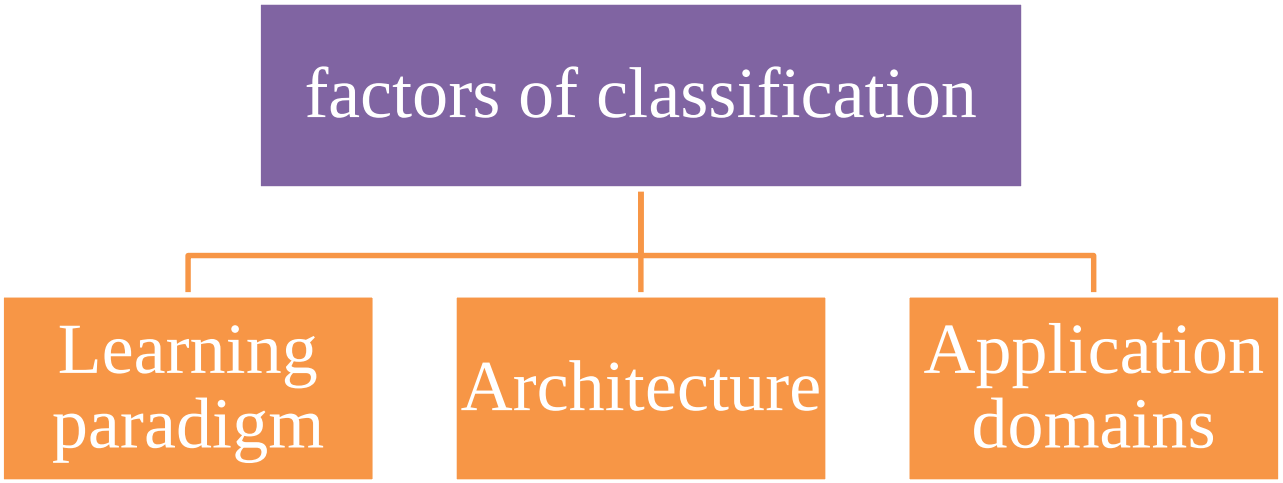
- Reinforcement learning is a machine learning paradigm in which an agent learns to interact with an environment in order to maximize a cumulative reward over time.



Different subfields of Machine Learning

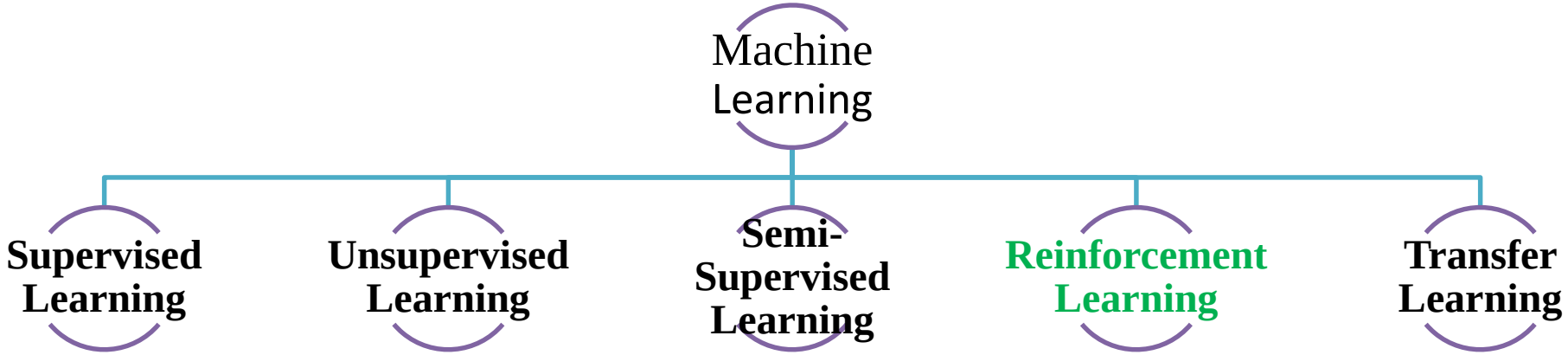
Different subfields of Machine Learning

- Machine Learning can be categorized into several subfields based on various factors.



Different subfields of Machine Learning : Learning paradigms

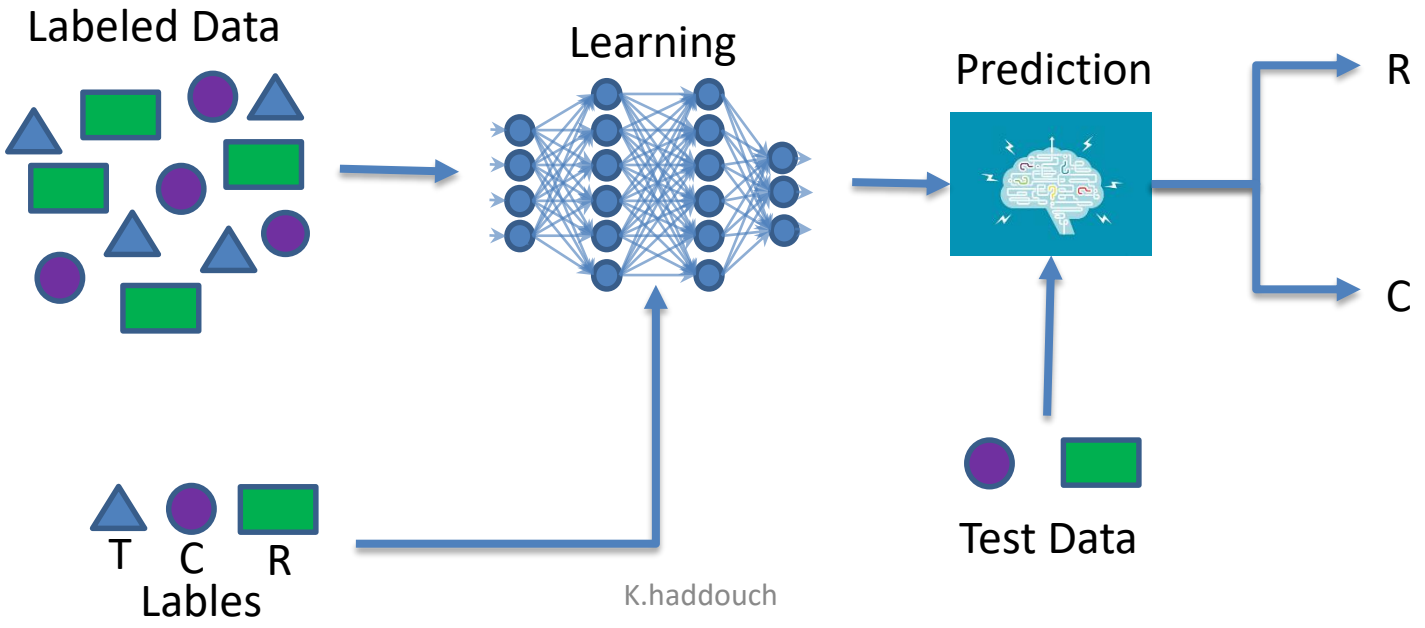
- There exist several types of learning paradigms in the field of Machine Learning (ML).
- Each one is used to handle a specific challenges in many field of applications.



Different subfields of Machine Learning : Learning paradigms

Supervised Learning:

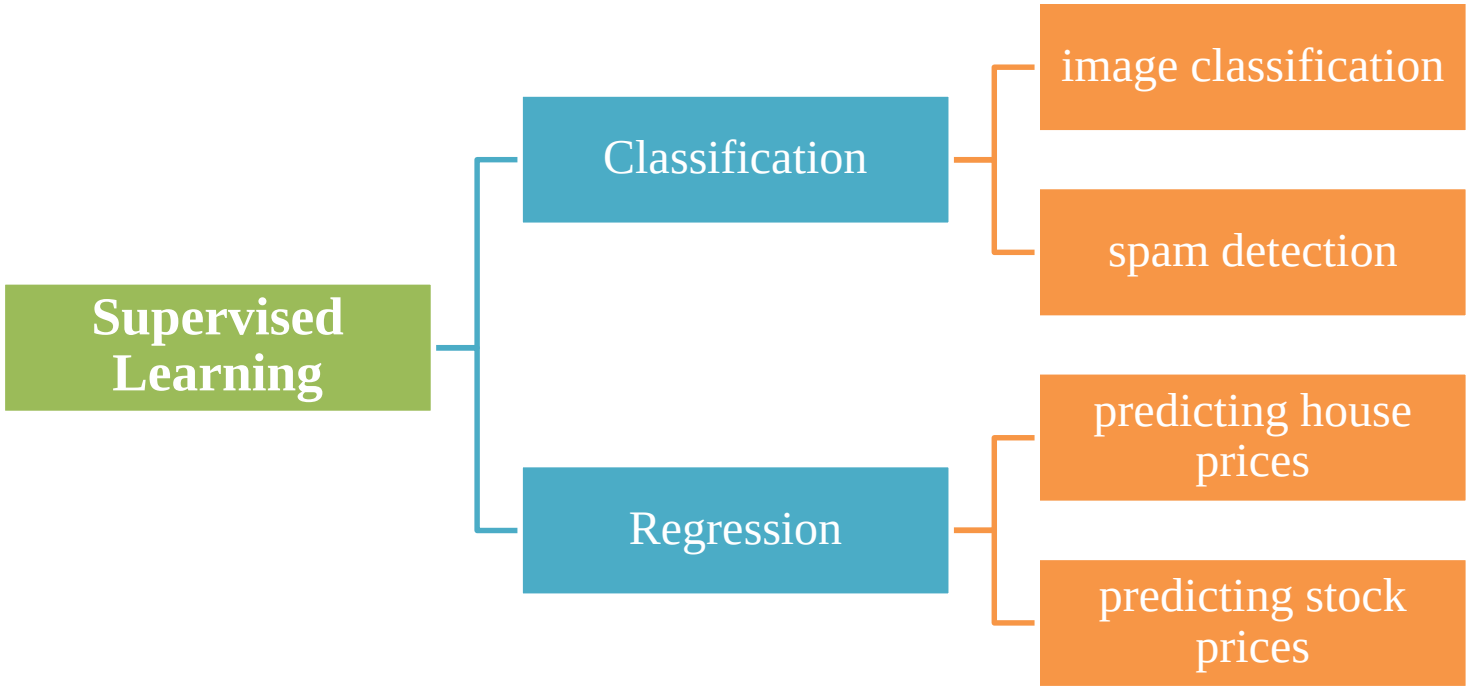
- Supervised learning involves training a model on a labeled dataset.
- Each input data point is associated with a corresponding target output.



Different subfields of Machine Learning : Learning paradigms

Supervised Learning:

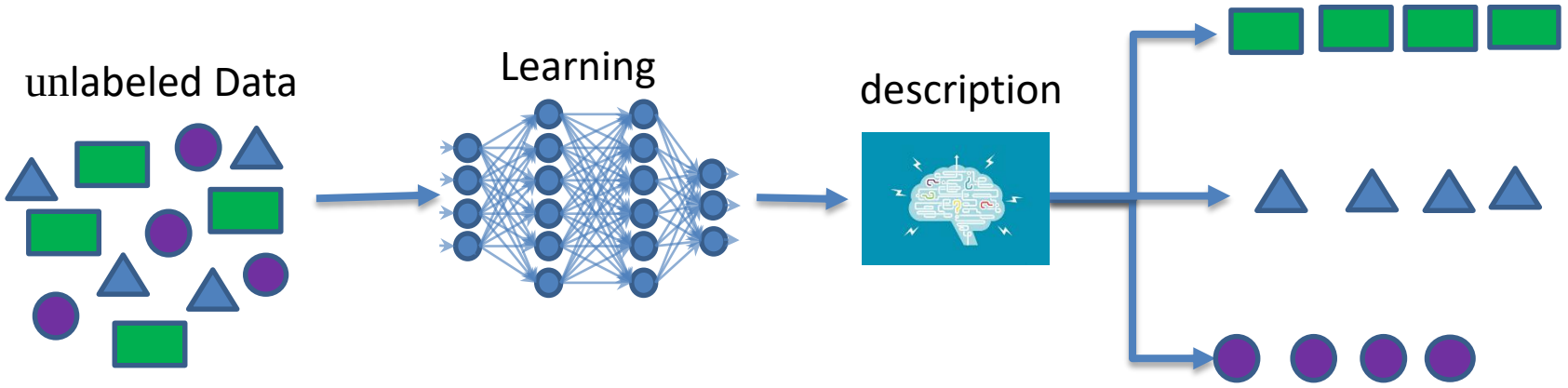
- The goal of supervised learning is to learn a mapping from input to output.



Different subfields of Machine Learning : Learning paradigms

Unsupervised Learning:

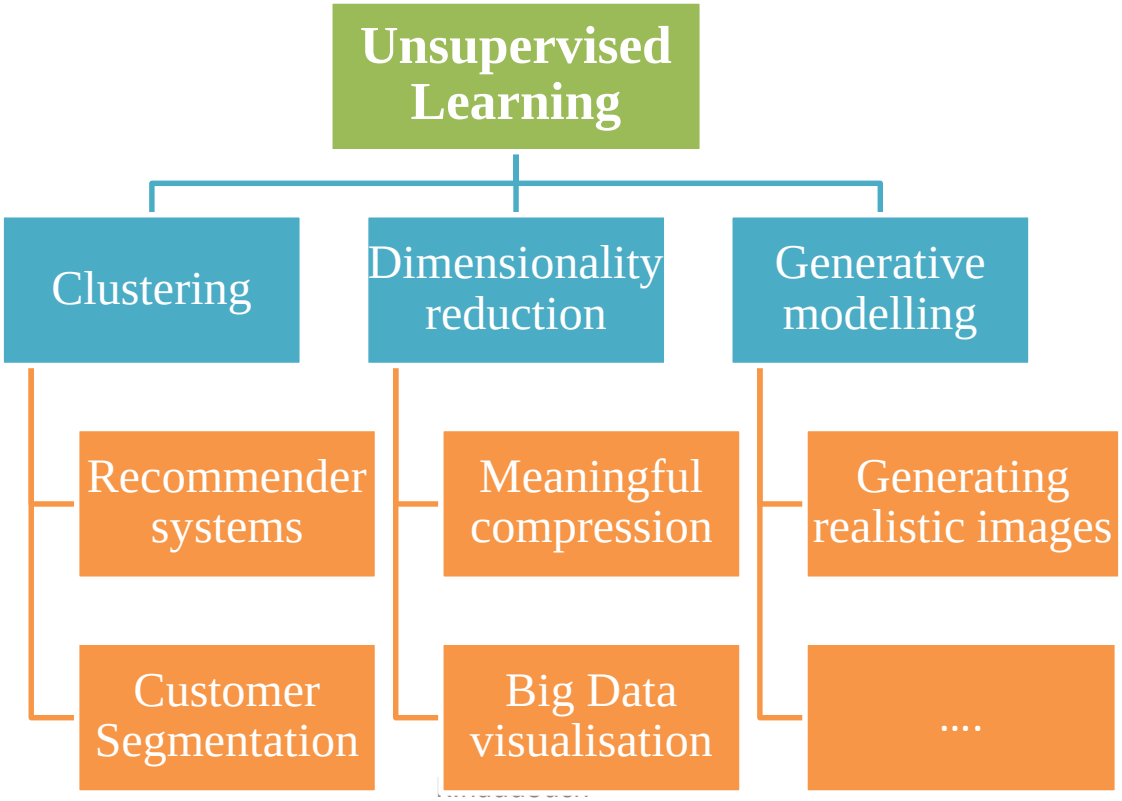
- Unsupervised learning involves training a model on an unlabelled dataset.
- This model consist to find hidden patterns in the data without explicit supervision.



Different subfields of Machine Learning : Learning paradigms

Unsupervised Learning:

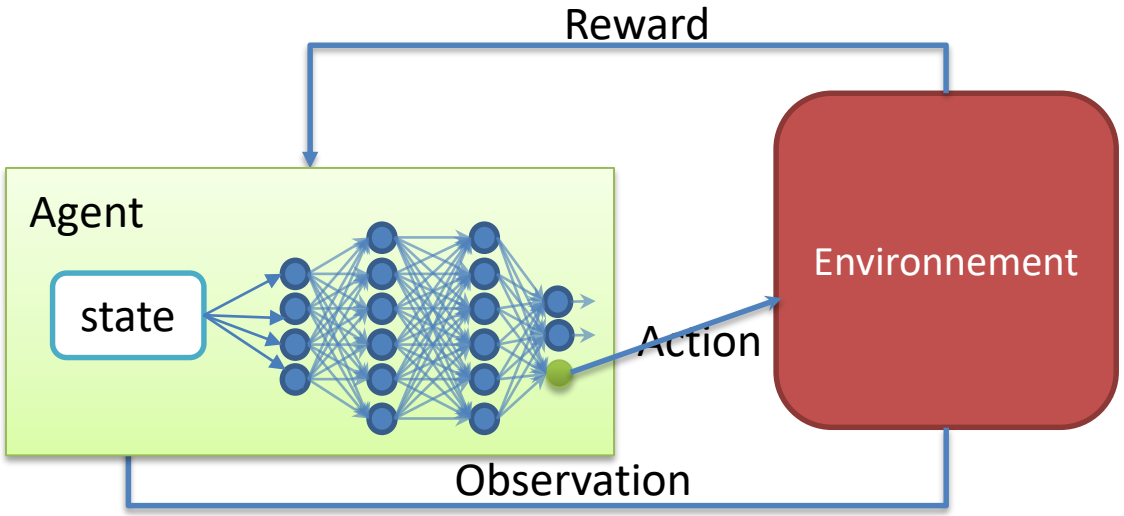
- The goal of unsupervised learning is to discover meaningful representations or clusters in the data.



Different subfields of Machine Learning : Learning paradigms

Reinforcement Learning:

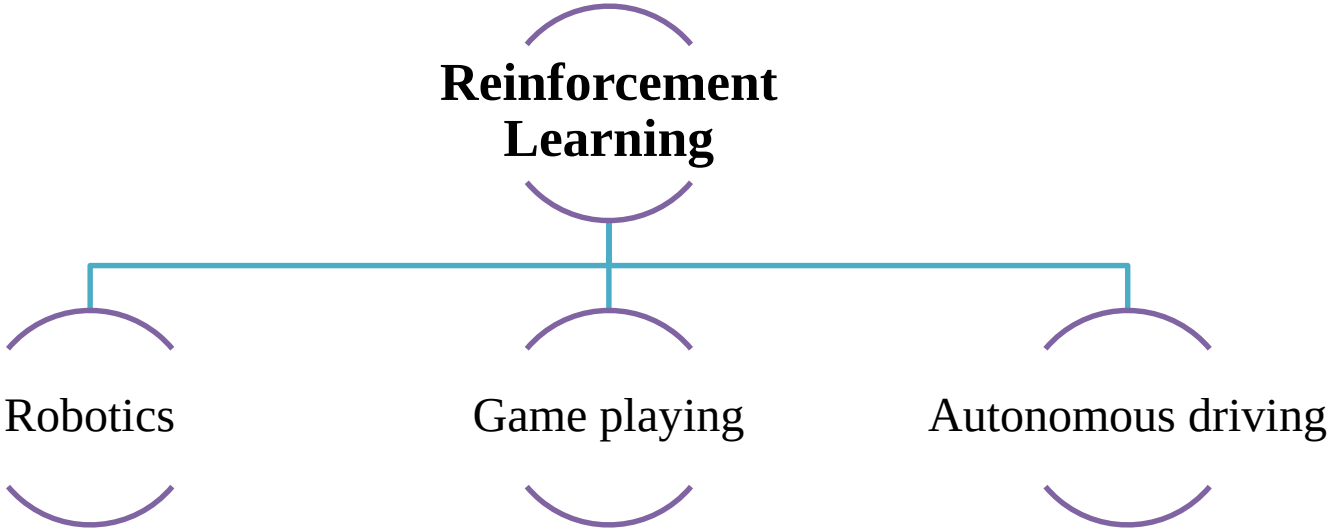
- Reinforcement learning involves training an agent to interact with an environment and learn optimal actions through trial and error.



Different subfields of Machine Learning : Learning paradigms

Reinforcement Learning:

- The agent receives feedback in the form of rewards or penalties based on its actions.
- After, its goal is to maximize the cumulative reward over time.



Different subfields of Machine Learning : Learning paradigms

Transfer Learning:

- Transfer learning involves leveraging knowledge acquired while training a model on one task to improve performance on a related task.
- Pre-trained models can be fine-tuned on a specific task with a smaller dataset to achieve better performance.
- Transfer learning is particularly useful in scenarios where :
  - Labeled data is scarce,
  - When training from scratch is computationally expensive.



Different subfields of Machine Learning : Learning paradigms

Semi-Supervised Learning:

- Semi-supervised learning combines elements of both supervised and unsupervised learning.
- This combination is obtained by leveraging a small amount of labelled data along with a larger pool of unlabelled data.
- The model learns from the labelled data to make predictions on the unlabelled data.
- Semi-supervised learning is particularly useful in scenarios where obtaining labelled data is expensive or time-consuming.

Different subfields of Machine Learning : Learning paradigms

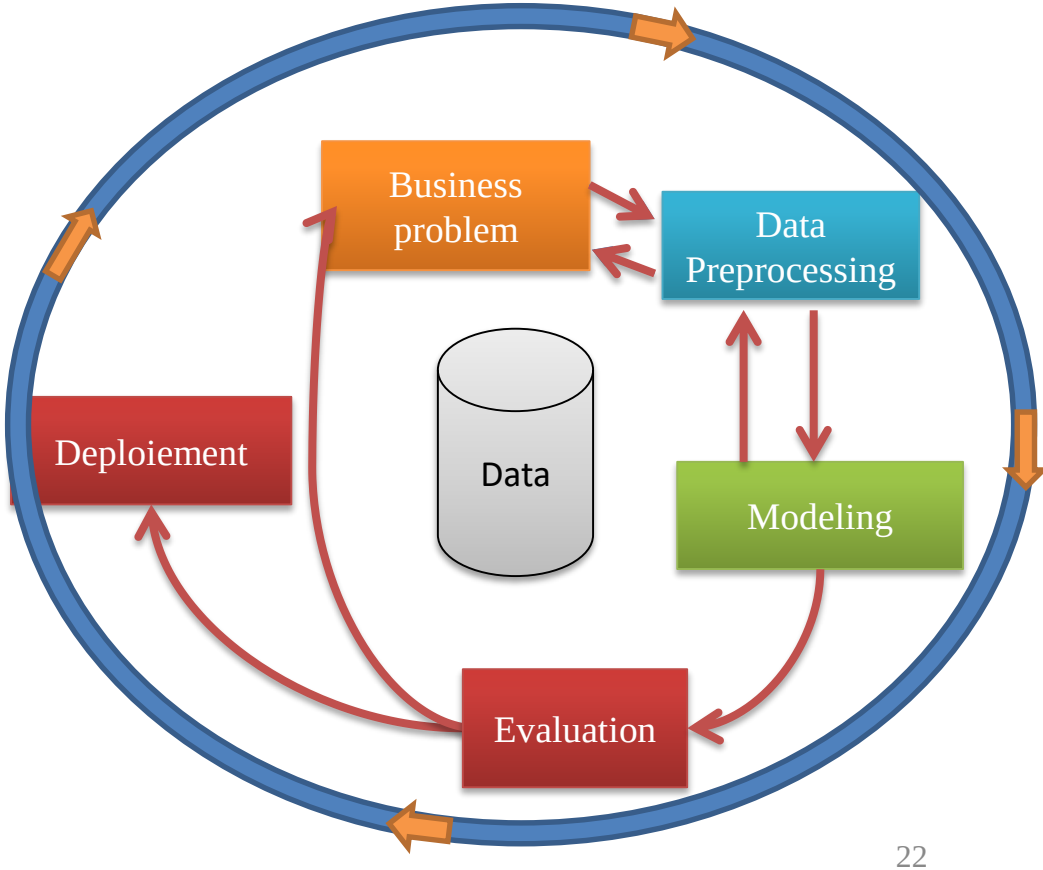
Comparison of learning paradigms

Learning Type	Input Data	Expected Output	Typical Example	Supervision
Supervised	(x, y)	Known label	Image recognition	Strong
Unsupervised	x only	Hidden structure	Clustering (K-means)	None
Reinforcement	(s, a, r)	Optimal policy	Autonomous agent	Delayed rewards
Semi-Supervised	Small set of labeled data + large set of unlabeled data	Improved classification model	Speech or handwriting recognition	Partial
Transfer Learning	Pretrained model + new task data	Adapted model for new domain	Fine-tuning a CNN for medical imaging	Indirect supervision

Training Machine Learning Models

Training Machine Learning Models

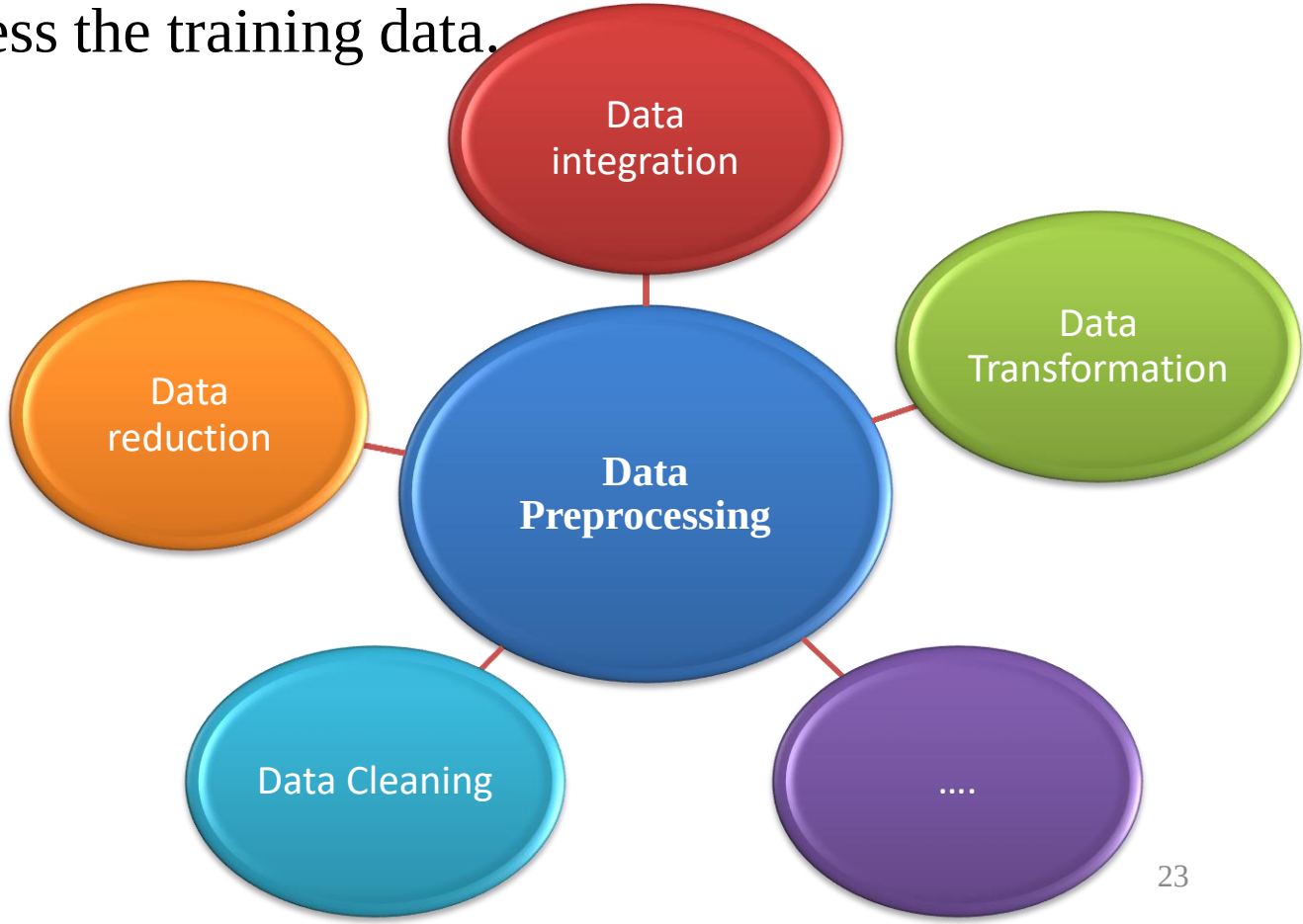
- Handling a reel problem via RL involves a series of phases:
  - **Definition the business problem,**
  - **Data Preprocessing,**
  - **Modeling,**
  - **Evaluation**,
  - **Deploiement**.



Training Machine Learning Models

Data Preprocessing:

- For teaching any machine learning element, the first phase is to pre-process the training data.



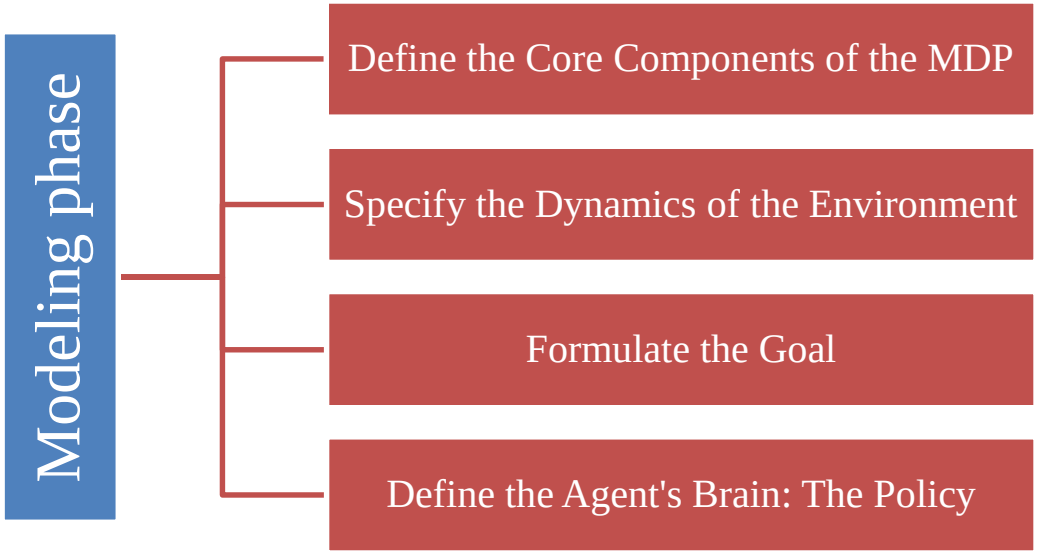
Training Machine Learning Models

- **Why Preprocess?**
  - Raw signals (images, sensors) can be noisy or redundant.
  - Preprocessing improves the stability and speed of learning.
- **Common Techniques:**
  - Normalization of observations.
  - Dimensionality reduction (PCA, ...).
  - State encoding (discretization of positions, ...).
  - Temporal filtering (sensor smoothing).
- **Example:** For a mobile robot, sensor values are normalized between [0,1] before being sent to the decision network.

Training Machine Learning Models

Modelling phase

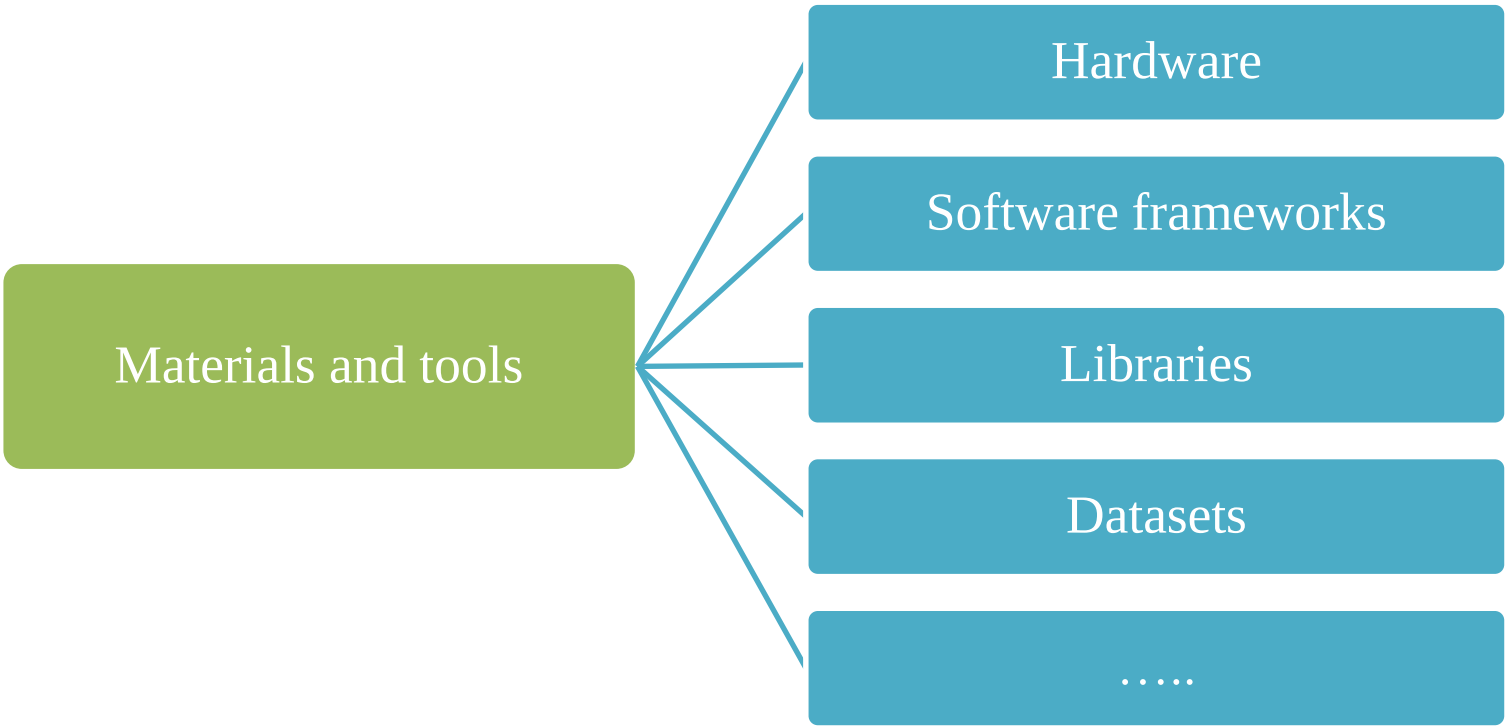
- This phase involves creating a mathematical model that represents the interaction between the agent and its environment.
- The standard model for this is a **Markov Decision Process (MDP)**.



Materials and tools for Reinforcement Learning

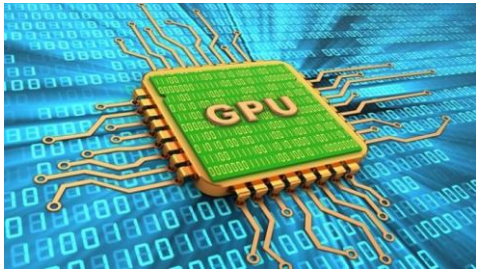
Materials and tools for Reinforcement Learning

- Reinforcement learning involve a wide range of resources.



Materials and tools for Reinforcement Learning

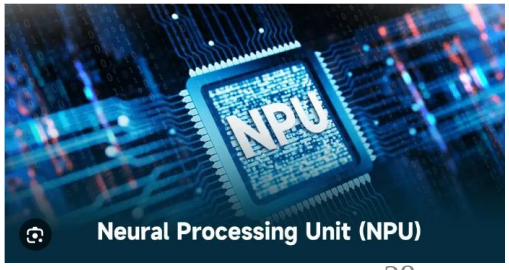
- **Graphics Processing Units (GPUs)**:
  - NVIDIA GPUs are commonly used for Reinforcement learning tasks.
  - This GPUs capable to ensure the parallel processing process.
  - It's for accelerate model training.



- **Tensor Processing Units(TPUs)**:
  - Developed by Google.
  - Specialized hardware accelerators.
  - Designed specifically for deep learning workloads,
  - Offering high performance and efficiency.



- **Neural Processing Units (NPUs)**
  - Is a specialized hardware accelerator,
  - Designed specifically for performing neural network computations efficiently.



Materials and tools for Reinforcement Learning

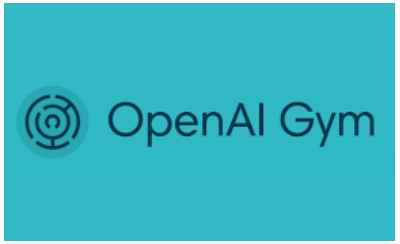
- **TensorFlow**:
  - An open-source reinforcement learning framework.
  - Developed by Google,
  - Widely used for building and training neural networks.



- **PyTorch**:
  - An open-source deep learning framework.
  - Developed by Facebook's AI Research lab (FAIR),
  - Known for its dynamic computation graph and ease of use.



- **Gymnasium / OpenAI Gym** - Simulation of RL environments
- **Stable-Baselines3** - Standard implementations (DQN, PPO, A2C, SAC...)



Materials and tools for Reinforcement Learning

- **NumPy**:
  - A fundamental library for numerical computing in Python,
  - Used for handling multidimensional arrays and mathematical operations.



- **Pandas**:
  - A powerful data manipulation and analysis library in Python,
  - Used for data preprocessing and manipulation tasks.



- **scikit-learn**:
  - A machine learning library in Python,
  - Offering simple and efficient tools for data mining and analysis.

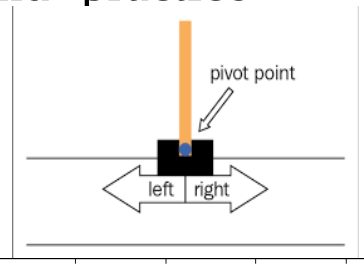


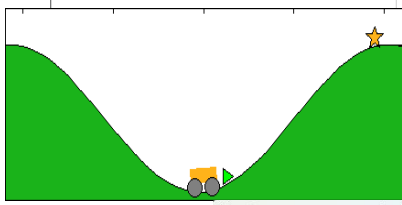
- **Matplotlib** :
  - Python librarie for data visualization,
  - Used for creating plots, charts, and graphs to visualize model performance and data distributions.



Materials and tools for Reinforcement learning : Dataset/environents

- Reinforcement Learning (RL) doesn't use static datasets like supervised learning.
- We need **environments** that generate data through interaction.
- These environments often come with their own built-in data or state representations.
- Here are examples of environments/datasets used to learn and practice Reinforcement Learning:
  - CartPole:
    - A pole is attached to a cart on a frictionless track.
    - The goal is to balance the pole by moving the cart left or right.
  - MountainCar:
    - A car is in a valley and must drive up the steep hill on either side.
    - It needs to build momentum by swinging back and forth.
  - FrozenLake :
    - A gridworld where you must navigate from the start to the goal without falling into holes.







Applications and Engineering Domains

Applications and Engineering Domains

Real-world RL applications:

Domain	Concrete Example
Robotics	Control of robotic arms, bipedal walking
Energy	Smart grid management
Transportation	Autonomous driving, traffic light optimization
Finance	Algorithmic trading, portfolio management
Healthcare	Automatic drug dosage, surgical assistance
Industry 4.0	Process control, predictive maintenance

Conclusion

Conclusion

Questions & answers

Chapter 2: Classical Algorithms for Reinforcement Learning

Introduction

- L'apprentissage par renforcement (RL) est une branche de l'apprentissage automatique.
- Dans laquelle un agent apprend à prendre des décisions en interagissant (action-réaction) avec un environnement.
- L'agent à pour objectif de maximiser une récompense cumulative en fonction du temps par essais et erreurs.
- Dans ce chapitre, nous nous concentrons sur les algorithmes classiques.

Eléments de RL

Eléments de RL

- **Agent :** Apprend et prend des actions.
- **Environnement :** Le monde avec lequel l'agent interagit.
- **État (s) :** Situation ou observation actuelle.
- **Action (a) :** Choix disponible pour l'agent.
- **Récompense (R) :** Retour d'information reçu après avoir effectué une action.
- **Politique (π) :** Stratégie qui définit la manière dont les actions sont choisies en fonction des états.
- **Fonction de valeur :** Mesure l'intérêt d'un état ou d'une action, pour une politique donnée.
- **Modèle :** Représentation de la dynamique de l'environnement.

Value-Based Methods

Value-Based Methods

- **Les méthodes basées sur la valeur** (Value-Based Methods) sont une catégorie d'algorithmes en apprentissage par renforcement où l'agent apprend à estimer la valeur des états ou des paires état-action.
- L'objectif est de construire une fonction de valeur optimale (comme la V-fonction ou Q-fonction) qui guide ensuite la politique de l'agent.
- L'agent met à jour iterativement ses estimations de valeur à travers ses interactions avec l'environnement, en utilisant typiquement des équations de type Bellman pour converger vers les valeurs optimales.

Value-Based Methods

- **Caractéristiques principales :**
  - Objectif : Apprendre la fonction de valeur optimale (V* ou Q*)
  - Fonctionnement : La politique est généralement dérivée indirectement de la fonction de valeur (par exemple, en choisissant l'action menant à l'état de plus grande valeur)
  - Avantage : Souvent plus stable et prévisible que les méthodes basées directement sur les politiques
  - Limite : Peut être moins efficace dans les espaces d'actions continus ou de grande dimension
- En général, elle se base sur deux méthode d'apprentissage:
  - Temporal Difference (TD) Learning
  - Méthodes de Monte Carlo

Value-Based Methods

Méthodes de Monte Carlo

- Apprennent les fonctions de valeur à partir d'épisodes complets.
- Idée : Calculer la moyenne des rendements (somme des récompenses actualisées) après chaque état ou chaque paire état-action.
- Ces méthodes sont basées sur l’expérience complète d’un agent dans un environnement.

Value-Based Methods

Méthodes de Monte Carlo

- Elles estiment les valeurs d'états ou d'actions à partir de plusieurs épisodes complets.
- Chaque épisode est une séquence :

$$S_0,A_0,R_1,S_1,A_1,R_2,...,S_T$$

- L'idée est de calculer le retour total à partir d'un état:

$$G_t=R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+...+\gamma^{T-t-1}R_T$$

- Puis, on estime la valeur moyenne pour chaque état :

$$V(s)=E[G_t|=s]$$

Value-Based Methods

Psodocode

Input:
π : politique à évaluer
γ : facteur d'actualisation ($0 \leq \gamma \leq 1$)
Initialize:
$V(s) \leftarrow 0$ for all $s \in S$
Returns(s) ← empty list for all $s \in S$
Repeat for each episode:
Generate an episode following policy π:
$(S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_T)$
For each state s appearing in the episode:
Find the first occurrence of s in the episode at time t
Compute the return:
$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots + \gamma^{T-t-1} R_T$
Append G_t to Returns(s)
Update the value estimate:
$V(s) \leftarrow$ average(Returns(s))

Value-Based Methods

Méthodes de Monte Carlo : Exemple

- Environnement :
  - Un agent navigue dans 3 états :
    A → B → C (terminal)
  - avec des récompenses observées lors de deux épisodes.

Épisode	Séquence (états et récompenses)	$\gamma = 1$
1	A (0) → B (+1) → C (0)	
2	A (0) → B (+2) → C (0)	

  - Épisode 1

Transition	Interprétation	Récompense R_{t+1}
A → B	L'agent quitte A et atteint B	$R_1=0$
B → C	L'agent quitte B et atteint C	$R_2=+1$
C → terminal	Fin de l'épisode	$R_3=0$

Value-Based Methods

Méthodes de Monte Carlo : Exemple

- Objectif : Calculer la valeur moyenne de chaque état.
  - Étape 1 : Calcul des retours (G)
    - Pour chaque épisode, on calcule le retour à partir du moment où on visite l'état. $G_t=R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+...+\gamma^{T-t-1}R_T$
    - Épisode 1 :

État	Récompenses futures	Calcul de G_t
A	$R_1+R_2+R_3=0+1+0$	1
B	$R_2+R_3=1+0$	1
C	$R_3=0$	0

    - Épisode 2 :

État	Récompenses futures	Calcul de G_t
A	$R_1+R_2+R_3=0+2+0$	2
B	$R_2+R_3=2+0$	2
C	$R_3=0$	0

Value-Based Methods

Méthodes de Monte Carlo : Exemple

- Objectif : Calculer la valeur moyenne de chaque état.
  - Étape 2 : Moyenne des retours par état

État	Retours observés	Moyenne V(s)
A	[1, 2]	(1 + 2) / 2 = **1.5**
B	[1, 2]	(1 + 2) / 2 = **1.5**
C	[0, 0]	**0**

Value-Based Methods

Approches pour estimer V(s)

- **Moyenne empirique (formule classique Monte Carlo)**
  - C'est la forme **originale** de la méthode de Monte Carlo dans le livre de Sutton & Barto.

$$V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_i(s)$$

où :

•N(s) = nombre total de fois où l'état *s* a été visité

•$G_i(s)$= retour obtenu lors de la i^e visite de *s*

Chaque fois qu'on visite un état, on **moyenne tous les retours** observés jusqu'à présent.

Value-Based Methods

Approches pour estimer V(s)

- Mise à jour incrémentale (forme pratique)
  - Pour éviter de recalculer la moyenne complète à chaque fois, on peut utiliser une formule récursive :

$$V(s) \leftarrow V(s) + \frac{1}{N(s)}[G - V(s)]$$

- C'est déjà une forme de moyenne incrémentale (où $\alpha=1/N(s)$).
- Et si on généralise encore, on obtient la règle avec un taux d'apprentissage constant :

$$V(s) \leftarrow V(s) + \alpha[G - V(s)]$$

Value-Based Methods

Méthodes de Monte Carlo : Résultat

V(A)=1.5,

V(B)=1.5,

V(C)=0

- Cela signifie que, en partant de A ou B, l'agent obtient une récompense totale de 1.5 comme moyenne avant la fin.

Value-Based Methods

Méthodes de Monte Carlo : Exemple en Python

```
import numpy as np
from collections import defaultdict

# Définition de l'environnement simple
episodes = [
    [("A", 0), ("B", 1), ("C", 0)],   # Episode 1
    [("A", 0), ("B", 2), ("C", 0)]    # Episode 2
]

gamma = 1.0  # facteur de discount
returns = defaultdict(list)
V = defaultdict(float)
```

Value-Based Methods

```
# Boucle sur chaque épisode
for episode in episodes:
    G = 0
    # On parcourt l'épisode à l'envers
    for t in reversed(range(len(episode))):
        state, reward = episode[t]
        G = gamma * G + reward
        # On enregistre le retour G_t pour cet état
        returns[state].append(G)

# Calcul des valeurs moyennes
for state in returns:
    V[state] = np.mean(returns[state])

# Affichage du résultat
for s, v in V.items():
    print(f"V({s}) = {v:.2f}")
```

Value-Based Methods

Méthodes de Monte Carlo : Exemple

- Avantages :
  - Simple à implémenter.
  - Ne nécessite pas de modèle de l'environnement.
  - Approche intuitive et statistique.
- Limites :
  - Besoin d'épisodes complets (ne convient pas aux tâches continues).
  - Convergence lente si les retours sont très variables.

Value-Based Methods

Temporal Difference (TD) Learning

- Les méthodes d'apprentissage temporel (TD) combinent des idées issues des méthodes de Monte Carlo et de la programmation dynamique.
- Elles mettent à jour la valeur d'un état sur la base de la valeur estimée de l'état suivant, sans attendre la fin de l'épisode.
- Règle de mise à jour TD(0)

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

α: learning rate (pas d'apprentissage)

γ: discount factor (facteur d'actualisation)

Value-Based Methods

Temporal Difference (TD) Learning

- Temporal Difference (TD) Learning est une famille de méthodes d'apprentissage par renforcement qui combinent :
  - Les idées des méthodes Monte Carlo (apprentissage par expérience),
  - Et des méthodes de Programmation Dynamique (utilisation des estimations courantes).
- L'idée clé :
  - On met à jour la valeur d'un état avant la fin de l'épisode, en utilisant une estimation du futur plutôt que la récompense finale.
- C'est ce qu'on appelle le bootstrapping :
- On améliore une estimation à partir d'une autre estimation.

Value-Based Methods

Temporal Difference (TD) Learning : Principe Fondamental

- On cherche à estimer la valeur d'un état V(s), qui représente la récompense cumulée attendue à partir de cet état :

$$V(s_t)=E[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\dots]$$

- Mais, contrairement à Monte Carlo, on ne va pas attendre la fin de l'épisode.
- On met à jour $V(s_t)$ dès qu'on observe une nouvelle récompense R_{t+1} et le nouvel état S_{t+1}.

Value-Based Methods

Temporal Difference (TD) Learning : Principe Fondamental

- Formule de mise à jour (TD(0))

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Où :
  - $V(S_t)$: valeur actuelle de l'état courant
  - R_{t+1} : récompense immédiate reçue
  - γ : facteur de discount ($0 \leq \gamma \leq 1$)
  - α : taux d'apprentissage ($0 < \alpha \leq 1$)
  - $V(S_{t+1})$: valeur estimée du prochain état

Value-Based Methods

Temporal Difference (TD) Learning : psodocode

Initialize V(s) arbitrarily for all states s ∈ S
V(terminal) = 0
Repeat (for each episode):
    Initialize state s
    Repeat (for each step in the episode):
        Choose action a from state s using policy π (e.g., ε-greedy)
        Take action a, observe reward r and next state s'
        # TD Update rule
        V(s) ← V(s) + α [R+ γ * V(s') - V(s)]
        s ← s'
    until s is terminal

Value-Based Methods

Temporal Difference (TD) Learning : Exemple

- Prenons un petit environnement : A → B → C (terminal)
- Récompenses :
  - A → B : +1
  - B → C : +2
- C : terminal (valeur 0)
- Paramètres :
  - $\alpha = 0.5$
  - $\gamma = 1$
- Initialement :
  - V(A) = 0, V(B) = 0, V(C) = 0

Value-Based Methods

Temporal Difference (TD) Learning : Exemple

- Étape 1 : Transition A → B

  $R_{t+1}=+1$

  $V(A) = V(A)+\alpha[R+\gamma V(B)-V(A)]$

  $V(A)=0+0.5[1+1\times 0-0]=0.5$

- Étape 2 : Transition B → C

  $R_{t+1}=+2$

  $V(B) = 0 + 0.5\,[2 + 1\times V(C) - 0] = 1.0$

- Étape 3 : C terminal

  Pas de mise à jour (état terminal)

Value-Based Methods

Temporal Difference (TD) Learning : Exemple

- Ces valeurs se mettront à jour au fil des épisodes jusqu'à converger vers les vraies valeurs attendues :

  $$V(A)=3, V(B)=2, V(C)=0$$

- si on répète indéfiniment, car 1 + 2 = 3 depuis A.

Value-Based Methods

Temporal Difference (TD) Learning : Exemple en python

```python
import numpy as np

# États et transitions
states = ["A", "B", "C"]
V = {s: 0.0 for s in states}  # valeurs initiales

alpha = 0.5
gamma = 1.0

# Simulation d'un épisode : A->B->C avec récompenses
episode = [("A", "B", 1), ("B", "C", 2)]
```

Value-Based Methods

Temporal Difference (TD) Learning : Exemple en python

```
# TD(0) Learning
for (s, s_next, r) in episode:
    V[s] = V[s] + alpha * (r + gamma * V[s_next] - V[s])

# Affichage des valeurs finales
for s, v in V.items():
    print(f"V({s}) = {v:.2f}")
```

Value-Based Methods

Temporal Difference (TD) Learning

- Interprétation :
  - Le TD(0) met à jour au fur et à mesure que l'agent avance.
  - Il n'a pas besoin d'attendre la fin d'un épisode pour apprendre.
  - Il apprend à prédire la récompense future d'un état à partir des transitions immédiates.
- Avantages :
  - Fonctionne pour des tâches continues (pas besoin de fin d'épisode)
  - Rapide (mise à jour après chaque étape)
  - Facile à implémenter
- Inconvénients :
  - Introduit un biais (car repose sur des estimations approximatives)
  - Convergence plus lente dans des environnements complexes

Value-Based Methods

comparaison

Méthode	Type	Nécessite fin d'épisode	Bootstrapping	Exemples
Monte Carlo	Échantillonnage complet	Oui	Non	Every-Visit, First-Visit
TD(0)	Estimation incrémentale	Non	Oui	Q-Learning, SARSA

Q-Learning

Q-Learning

- Le Q-Learning est l'un des algorithmes d'apprentissage par renforcement les plus populaires.
- Le Q-Learning est basé sur TD Learning avec une politique off-policy.
- Il apprend la valeur des actions dans chaque état, appelée Q-valeur :

$$Q(s,a) \approx \text{valeur attendue du retour en prenant l'action a dans l'état}$$

- Dans TD Learning, on met à jour les valeurs après chaque transition (pas besoin d'attendre la fin de l'épisode).

Q-Learning

- La mise à jour de base (pour les états) est :

$$V(s) \leftarrow V(s) + \alpha[R + \gamma V(s') - V(s)]$$

- Cet algorithme apprend directement la fonction de valeur-action Q(s,a).
- Règle de mise à jour :

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

- où :
  - s : État courant
  - a : Action effectuée
  - R: Récompense reçue
  - s' : État suivant
  - α : Taux d'apprentissage
  - γ : Facteur de discount
  - $\max_{a'} Q(s',a')$: Action la plus prometteuse dans le nouvel état

Q-Learning

- Propriétés clés de l'algorithme Q-Learning:
  - Hors politique : L'agent apprend à partir d'actions qui ne sont pas nécessairement celles sélectionnées par la politique courante (par exemple, une politique ε-greedy).
  - Converge vers la fonction optimale Q* sous certaines conditions.
- L'agent peut explorer avec une politique différente (par ex. ε-greedy),
- mais il met à jour ses Q-valeurs comme s'il suivait la meilleure politique (greedy).
- Autrement dit, il agit selon une politique d'exploration (par ex. parfois aléatoire),
- Mais apprend en supposant qu'il choisira la meilleure action à l'avenir.

Q-Learning

Psodocode

Initialize Q(s, a) arbitrarily for all states s ∈ S, actions a ∈ A(s)
Q(terminal, ·) = 0
Repeat (for each episode):
  Initialize state s
  Repeat (for each step in the episode):
    Choose action a from state s using policy derived from Q (e.g., ε-greedy)
    Take action a, observe reward r and next state s'
    // Q-Learning update rule
    Q(s, a) ← Q(s, a) + α [R+ γ * max_a' Q(s', a') - Q(s, a)]
    s ← s'
  until s is terminal

Q-Learning

Exemple

- Imaginons un mini-environnement simple :
  - États : A, B, C (terminal)
  - Actions possibles : Left (L), Right (R)
- Récompenses :
  - De A → B (Right) : +1
  - De B → C (Right) : +2
  - De A → C (Left) : 0
- Paramètres :
  - $\alpha=0.5, \gamma=0.9$
  - Au début : $Q(s,a)=0$ pour tous s,a

Q-Learning

Exemple

Épisode : A (Right) → B (Right) → C

Étape	(s,a,r,s')	Calcul de la mise à jour
1	(A, Right, +1, B)	$Q(A,R) \leftarrow 0+0.5[1+0.9 \max Q(B,*)-0]$ $Q(A,R) \leftarrow 0.5 \times 1=0.5$
2	(B, Right, +2, C)	$Q(B,R) \leftarrow 0+0.5[2+0.9 \times 0-0]$ $Q(B,R) \leftarrow 0.5 \times 2=1.0$

Q-Learning

Exemple en python

```python
import numpy as np
import random

# Définition de l'environnement simplifié
states = ['A', 'B', 'C']
actions = ['Left', 'Right']

# Initialisation Q(s,a)
Q = {s: {a: 0.0 for a in actions} for s in states}

# Paramètres
alpha = 0.5
gamma = 0.9
epsilon = 0.2
```

Q-Learning

```
def get_reward(s, a):
    if s == 'A' and a == 'Right':
        return 1, 'B'
    elif s == 'B' and a == 'Right':
        return 2, 'C'
    elif s == 'A' and a == 'Left':
        return 0, 'C'
    else:
        return 0, 'C'
```

Q-Learning

```
# Entraînement
for episode in range(100):
    state = 'A'
    while state != 'C':
        # Politique ε-greedy
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = max(Q[state], key=Q[state].get)

        reward, next_state = get_reward(state, action)

        # Calcul du Q-learning
        best_next = max(Q[next_state].values()) if next_state != 'C' else 0
        Q[state][action] = Q[state][action] + alpha *
                           (reward + gamma * best_next - Q[state][action])

        state = next_state
```

Q-Learning

```
# Affichage des valeurs apprises
for s in Q:
    print(s, Q[s])
```



```
A {'Left': 0.0, 'Right': 2.8}
B {'Left': 0.0, 'Right': 2.0}
C {'Left': 0.0, 'Right': 0.0}
```

L'agent apprend progressivement que :

- **À A**, l'action *Right* est la meilleure
- **À B**, l'action *Right* mène à une plus grande récompense

SARSA (On-policy TD Control)

SARSA (On-policy TD Control)

- SARSA et Q-Learning se ressemblent presque totalement.
- mais leur philosophie d'apprentissage est différente :
  - SARSA est on-policy,
  - Q-Learning est off-policy.
- SARSA est un algorithme d'apprentissage par renforcement basé sur la méthode Temporal Difference (TD),
- Utilisé pour apprendre une politique d'action.

SARSA (On-policy TD Control)

- Le nom SARSA vient des 5 éléments utilisés à chaque mise à jour : (s,a,r,s',a')
- Où
  - s : état courant
  - a : action effectuée
  - r : récompense obtenue
  - s' : état suivant
  - a' : action choisie dans s' selon la même politique
- La règle d'apprentissage :

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma Q(s',a') - Q(s,a)]$$

SARSA (On-policy TD Control)

- **Exemple**
  - Prenons le même environnement que pour le Q-Learning :
  - États : A, B, C (terminal)
  - Actions : Left (L), Right (R)
  - Récompenses :
    - $A \rightarrow B$ (Right) = +1
    - $B \rightarrow C$ (Right) = +2
    - $A \rightarrow C$ (Left) = 0
  - Paramètres :
    - $\alpha=0.5, \gamma=0.9$
    - Et au départ : $Q(s,a)=0$

SARSA (On-policy TD Control)

- **Exemple**
  - Épisode (généré par la politique ε-greedy) :
  - A (Right, r=+1) → B (Right, r=+2) → C (terminal)
  - Transition 1 :

    $(s,a,r,s',a')=(A,R,+1,B,R)$
    - Mise à jour :

      $Q(A,R)=0+0.5[1+0.9\times Q(B,R)-0]=0.5\times 1=0.5$
  - Transition 2 :
    - $(s,a,r,s',a')=(B,R,+2,C,-)$
    - $Q(B,R)=0+0.5[2+0.9\times 0-0]=1.0$

SARSA (On-policy TD Control)

- **Exemple**
  - Après ce premier épisode :
    Q(A,R)=0.5
    Q(B,R)=1.0
  - Si on répète plusieurs épisodes, les Q-valeurs convergent vers :
    Q(A,R)≈2.7
    Q(B,R)≈2
  - Ces valeurs sont similaires à celles obtenues avec Q-learning, mais SARSA peut être plus prudent si la politique ε-greedy continue d'explorer.

SARSA (On-policy TD Control)

Psodocode

Initialize Q(s, a) arbitrarily for all states $s \in S$, actions $a \in A(s)$
Q(terminal, ·) = 0
Repeat (for each episode):
  Initialize state s
  Choose action a from s using policy derived from Q (e.g., ε-greedy)
  Repeat (for each step in the episode):
    Take action a, observe reward r and next state s'
    Choose next action a' from s' using policy derived from Q (e.g., ε-greedy)
    //SARSA update rule
    $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma * Q(s', a') - Q(s, a)]$
    $s \leftarrow s'$
    $a \leftarrow a'$
  until s is terminal

Q-Learning and SARSA

```
# Entraînement
for episode in range(100):
    state = 'A'
    # Choisir la première action
    if random.random() < epsilon:
        action = random.choice(actions)
    else:
        action = max(Q[state], key=Q[state].get)
    while state != 'C':
        reward, next_state = get_reward(state, action)
        # Choisir la prochaine action selon la même politique (on-policy)
        if next_state != 'C':
            if random.random() < epsilon:
                next_action = random.choice(actions)
            else:
                next_action = max(Q[next_state], key=Q[next_state].get)
        else:
            next_action = None
        # Mise à jour SARSA
        target = reward + gamma * Q[next_state][next_action]
         if next_action else reward
        Q[state][action] = Q[state][action] + alpha * (target - Q[state][action])
        # Avancer
        state, action = next_state, next_action
```

Q-Learning and SARSA

```
# Affichage
for s in Q:
    print(s, Q[s])
```



```
A {'Left': 0.0, 'Right': 2.799560546872135}
B {'Left': 0.0, 'Right': 1.9999999999998863}
C {'Left': 0.0, 'Right': 0.0}
```

Q-Learning and SARSA

- **Analyse :**
  - L'agent apprend progressivement la meilleure action dans chaque état.
  - Il agit et apprend avec la même politique (ε-greedy).
  - Si la politique explore beaucoup (grand ε), SARSA sera plus prudent.
  - Si $\varepsilon \rightarrow 0$, SARSA $\approx$ Q-Learning.

Q-Learning and SARSA

Différence entre Q-Learning SARSA (On-policy TD Control)

Aspect	**SARSA (On-policy)**	**Q-Learning (Off-policy)**
Mise à jour	$R+\gamma Q(s',a')$	$R+\gamma \max_{a'} Q(s',a')$
Politique utilisée pour agir	La même que celle utilisée pour apprendre	Politique “greedy” idéale
Comportement	Plus conservateur, prend en compte les risques liés à l’exploration	Plus optimiste, suppose qu’on prendra toujours la meilleure action
Type	On-policy	Off-policy

La politique (π)

La politique (π)

- En apprentissage par renforcement, la politique π(policy en anglais) décrit la manière dont l'agent choisit ses actions en fonction de l'état courant :

$$\pi(a|s)=P(A_t=a|S_t=s)$$

- Autrement dit, la politique π est une stratégie de comportement.
- Elle définit la probabilité de choisir une action *a* lorsqu'on se trouve dans un état *s*.

La politique (π)

- **Types de politiques**
  - Il existe deux grandes catégories de politiques selon leur nature :

Type	Description	Exemple
Déterministe	Une seule action est choisie pour chaque état	$\pi(s)=a$
Stochastique (probabiliste)	Les actions sont choisies selon une distribution de probabilité	$\pi(a/s)=P(A_t=a/S_t=s)$

  - Remarque : En pratique, dans les environnements d'exploration, on préfère souvent les politiques stochastiques pour éviter de tomber dans des comportements figés.

La politique (π)

- Les principales stratégies qu'un agent peut utiliser pour équilibrer exploration et exploitation :
  - **Greedy Policy (politique avide)**
  - **ε-Greedy Policy (epsilon-greedy)**
  - **ε-decay Policy**
  - **Softmax (Boltzmann Exploration)**
  - **Upper Confidence Bound (UCB)**
  - **…**

La politique (π)

Greedy Policy (politique avide)

- L'agent choisit toujours l'action ayant la plus grande valeur estimée *Q(s,a)*.

$$a = argmax_a Q(s,a)$$

- Avantage : Exploite la connaissance actuelle.
- Inconvénient : N'explore jamais de nouvelles actions, risque de rester bloqué dans un optimum local.

La politique (π)

ε-Greedy Policy (epsilon-greedy)

- L'agent choisit la meilleure action (greedy) avec probabilité $1-\varepsilon$
- Et choisit une action aléatoire (exploration) avec probabilité ε

$$a = \begin{cases} \text{aléatoire} & \text{avec probabilité } \varepsilon \\ \arg\max_a Q(s,a) & \text{avec probabilité } 1-\varepsilon \end{cases}$$

- Typiquement : $\varepsilon=0.1$ (10 % du temps, on explore)
- Avantage : Bon compromis entre exploration et exploitation désavantage : Le taux ε fixe peut être sous-optimal (d'où les variantes adaptatives)

La politique (π)

ε-decay Policy

- Une variante du précédent.
- ε diminue progressivement au fil du temps :

$$\varepsilon_t = \varepsilon_0 \times e^{-kt}$$

- Cela permet :
  - D'explorer beaucoup au début
  - Puis de se stabiliser sur la meilleure politique à la fin

La politique (π)

Softmax (Boltzmann Exploration)

- Au lieu de choisir la meilleure action ou une aléatoire, on choisit probabilistiquement selon les valeurs Q :

$$P(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_b e^{Q(s,b)/\tau}}$$

- où τ est la température (contrôle l'exploration) :
  - Si τ est grande alors l'exploration élevée (toutes les actions probables)
  - Si τ est faible alors comportement plus "greedy"
- Avantage : Exploration douce et contrôlée
- Limite : Nécessite un calcul exponentiel (plus coûteux)

La politique (π)

Choisir une politique selon le contexte

Contexte	Politique adaptée
Entraînement initial	ε-greedy avec ε élevé ou Softmax avec grande température
Phase d'apprentissage avancée	ε-decay ou Softmax avec petite température
Exploitation finale (production)	Greedy (maximisation pure)

conclusion

Chapter 3 : Advanced Reinforcement Learning

Environnements d'Apprentissage par Renforcement (Gym / Gymnasium)

Introduction

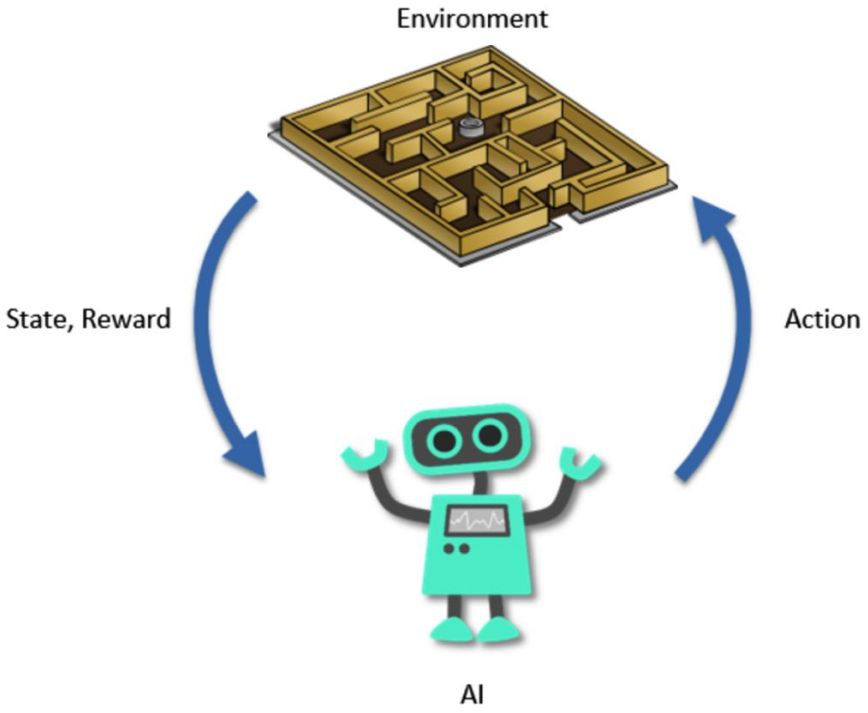
- Nous avons vue que la bibliothèque Gym propose un environnement pour entraîner l'agent d'apprentissage par renforcement pour FrozenLake.
- Il faut noter que Gym propose d’autres nombreux environnements (CartPole, MountainCar, etc.),
- Pour comprendre clairement comment un environnement Gym est conçu, nous allons commencer par l'environnement de base.
- Par la suite, nous comprendrons quelque les autres environnements Gym plus complexes.

Introduction

- Prenons l'un des environnements les plus simples, appelé "Frozen Lake".
- Comme nous pouvons l'observer, dans Frozen Lake, l'objectif de l'agent est de partir de l'état initial S et d'atteindre l'état but G.
- Or, la création d’un environnement personnalisé est utile lorsque :
  - on veut résoudre un problème spécifique du monde réel ;
  - on veut tester un nouvel algorithme ;
  - on veut créer un TP ou un challenge pour les étudiants ;
  - on veut simuler un processus (file d’attente, robot, finance, contrôle industriel…).

Introduction

- Un environnement Gym est un **cadre standardisé** qui permet à un agent RL d'interagir avec un système selon le paradigme :
  - **État**
  - **Action**
  - **Nouvel état**
  - **Récompense**



Structure d’un environnement Gym

Structure d’un environnement Gym

- Un environnement Gym doit obligatoirement définir les fonctions suivantes :
  - action_space
  - observation_space
  - reset
  - Step
  - close
  - Render (optionnel)

Structure d'un environnement Gym

- action_space :
  - Définit les actions possibles que l'agent peut exécuter.
  - Peut être discret (ex: haut, bas, gauche, droite) ou continu (ex: angle de rotation).
  - Exemple en python :
    - Création de l'environnement FrozenLake

```
env = gym.make("FrozenLake-v1", render_mode="human")
```

    - Afficher l'espace d'actions

```
print("Action space:", env.action_space)
print("Actions possibles:", env.action_space.n)
print("Exemple d'action aléatoire:", env.action_space.sample())
```

    - Signification des actions: 0 = LEFT (gauche), 1 = DOWN (bas), 2 = RIGHT (droite), 3 = UP (haut)
    - Vérification si une action est valide

```
print("L'action 5 est-elle valide?", 5 in env.action_space)
```

Structure d'un environnement Gym

- action_space :
  - Code :

```python
import gymnasium as gym
env = gym.make("FrozenLake-v1", render_mode="human")
print("espace d'action :", env.action_space)
print("Actions possibles:", env.action_space.n)
print("Exemple d'une action aléatoire:", env.action_space.sample())
print("L'action 5 est-elle valide ou non ?", 5 in env.action_space)
```



```
espace d'action : Discrete(4)
Actions possibles: 4
Exemple d'une action aléatoire: 2
L'action 5 est-elle valide ou non ? False
```

Structure d'un environnement Gym

- observation_space :
  - Type d'états fournis par l'environnement.
  - Définit la structure des informations que l'agent reçoit de l'environnement.
  - Peut être discret (ex: position sur une grille) ou continu (ex: coordonnées).
  - Exemple en python :
    - Les observations Discrete(16) - signifie 16 états discrets (0 à 15):

      ```
      print("Observations dans l'espace:", env.observation_space)
      ```
    - Le nombre d'états (16):

      ```
      print("Nombre d'états:", env.observation_space.n)
      ```
    - Exemple d'observation

      ```
      print("Exemple d'observation:", env.observation_space.sample())
      ```
    - Dans cet exemple de FrozenLake, les 16 cases sont numérotées de 0 à 15 (ligne par ligne) avec S=0 (départ), G=15 (but), H=[5,7,11,12] (trous), F=autres cases

Structure d’un environnement Gym

- observation_space :
  - Code :

```
print("Observations dans l'espace:", env.observation_space)
print("Nombre d'états:", env.observation_space.n)
print("Exemple d'observation:", env.observation_space.sample())
```

```
Observations dans l'espace: Discrete(16)
Nombre d'états: 16
Exemple d'observation: 11
```

Structure d'un environnement Gym

- reset() :
  - Réinitialise l'environnement à un état initial (au début d'un épisode).
  - Exemple
    - Réinitialisation de l'environnement

```
observation, info = env.reset()
```

observation: l'état initial (toujours 0 pour FrozenLake)

info: contient des informations supplémentaires (dictionnaire)

    - Etat initial après la réinitialisation :

```
print(f"État initial après reset: {observation}")
print(f"Info: {info}")
```

    - La reproduction avec seed

```
observation, info = env.reset(seed=42)
print(f"État avec seed 42: {observation}")
```

Structure d’un environnement Gym

- reset() :
  - Code

```
observation, info = env.reset()
print(f"État initial après reset: {observation}")
print(f"Info: {info}")
observation, info = env.reset(seed=42)
print(f"État avec seed 42: {observation}")
```

```
État initial après reset: 0
Info: {'prob': 1}
État avec seed 42: 0
```

Structure d’un environnement Gym

- step(action): Exécute une action et retourne un résultat:
  - observation: Le nouvel état de l'environnement
  - reward: La récompense obtenue
  - terminated: Booléen indiquant si l'épisode est terminé (état terminal atteint)
  - truncated: Booléen indiquant si l'épisode a été interrompu (limite de temps)
  - info: Informations supplémentaires pour le débogage

Structure d’un environnement Gym

- step(action): Exécute une action et retourne un résultat:
  - Exemple
    - Initialisation d’une action : action = 2 # RIGHT
    - Calculer le résultats

      observation, reward, terminated, truncated, info = env.step(action)
    - Affichage

```
print(f"Nouvel état après action {action}: {observation}")
print(f"Récompense: {reward}")
print(f"Terminé: {terminated}")
print(f"Tronqué: {truncated}")
print(f"Info: {info}")
```

Structure d'un environnement Gym

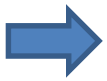
- step(action): Exécute une action et retourne un résultat:
  - Exemple :

```
observation, info = env.reset()
total_reward = 0
done = False
while not done:
        action = env.action_space.sample()  # réaliser une action aléatoire
        observation, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        done = terminated or truncated
print(f"Récompense totale: {total_reward}")
```

Structure d'un environnement Gym

- step(action): Exécute une action et retourne un résultat:
  - Code :

```python
action = 2
observation, reward, terminated, truncated, info = env.step(action)
print(f"Nouvel état après action {action}: {observation}")
print(f"Récompense: {reward}")
print(f"Terminé: {terminated}")
print(f"Tronqué: {truncated}")
print(f"Info: {info}")
observation, info = env.reset()
total_reward = 0
done = False
while not done:
    action = env.action_space.sample()
    observation, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    done = terminated or truncated
print(f"Récompense totale: {total_reward}")
```



```
Nouvel état après action 2: 5
Récompense: 0
Terminé: True
Tronqué: False
Info: {'prob': 1.0}
```

Structure d’un environnement Gym

- close() : Libération des ressources utilisées par l'environnement.
  - Exemple :
    - Utilisation standard

```
env = gym.make("FrozenLake-v1", render_mode="human")
try:
        # Utiliser l'environnement
        observation, info = env.reset()
finally:
        # Toujours fermer pour libérer les ressources
        env.close()
        print("Environnement fermé proprement")
# Avec un contexte (recommandé)
```

Structure d’un environnement Gym

- close() : Libération des ressources utilisées par l'environnement.
  - Code :

```python
env = gym.make("FrozenLake-v1", render_mode="human")
try:
    observation, info = env.reset()
finally:
    env.close()
    print("Environnement fermé proprement")
```

```
Environnement fermé proprement
```

Création d’un environnement personnalisé

Création d'un environnement personnalisé

- Créer un environnement d'apprentissage par renforcement (RL) revient à concevoir un jeu vidéo ou une simulation.
- Avant d'écrire la moindre ligne de code, il est essentiel de bien modéliser le problème d'apprentissage à résoudre.
- Cette phase de modélisation est cruciale : un environnement mal conçu rendra l'apprentissage difficile, voire impossible.
- Questions clés pour la modélisation
  - Quelle compétence l'agent doit-il acquérir ?
    - Naviguer dans un labyrinthe ?
    - Équilibrer et contrôler un système ?
    - Optimiser l'allocation des ressources ?
    - Jouer à un jeu stratégique ?

Création d'un environnement personnalisé

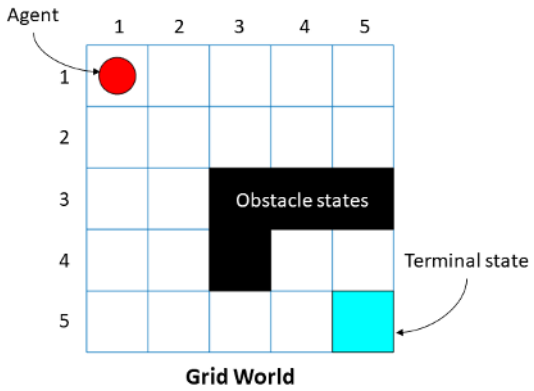
- Questions clés pour la modélisation
  - De quelles informations l'agent a-t-il besoin ?
    - Position et vitesse ?
    - État actuel du système ?
    - Données historiques ?
    - Observabilité partielle ou totale ?
  - Quelles actions l'agent peut-il entreprendre ?
    - Choix discrets (monter/descendre/gauche/droite) ?
    - Contrôle continu (angle de braquage, accélération) ?
    - Actions multiples simultanées ?
  - Comment mesure-t-on le succès ?
    - Atteindre un objectif précis ?
    - Minimiser le temps ou l'énergie ?
    - Maximiser un score ?
    - Éviter les échecs ?

Création d'un environnement personnalisé

- Questions clés pour la modélisation
  - Quand les épisodes doivent-ils se terminer ?
    - Achèvement de la tâche (succès/échec) ?
    - Limites de temps ?
    - Contraintes de sécurité ?
- Pour nos exemples du code dans cette partie, nous allons créer un environnement GridWorld simple :
  - Compétence : Naviguer efficacement jusqu'à une cible
  - Information : Position de l'agent et position de la cible sur une grille
  - Actions : Se déplacer vers le haut, le bas, la gauche ou la droite
  - Succès : Atteindre la cible en un minimum de déplacements
  - Fin : Lorsque l'agent atteint la cible (ou limite de temps optionnelle)

Création d’un environnement personnalisé

- Cet exemple propose un problème d’apprentissage clair, suffisamment simple pour être compris, mais complexe à résoudre de manière optimale.
- On cherche à implémenter le jeu GridWorld sous la forme d'une grille carrée bidimensionnelle de taille fixe.
- L'agent peut se déplacer verticalement ou horizontalement entre les cellules de la grille à chaque étape.
- Le but est d'atteindre une cible placée aléatoirement au début de l'épisode.



Création d’un environnement personnalisé

Environnement __init__

- Comme tous les environnements, notre environnement personnalisé héritera de gymnasium.Env, qui définit la structure que tous les environnements doivent respecter.
- L'une des exigences est la définition des espaces d'observation et d'action, qui déclarent quelles entrées (actions) et sorties (observations) sont valides pour cet environnement.
- Comme indiqué dans notre conception, notre agent dispose de quatre actions discrètes (déplacement dans les directions cardinales), nous utiliserons donc l'espace Discrete(4).

Création d’un environnement personnalisé

Environnement __init__

- Pour les observations, plusieurs options s'offrent.
- Nous pourrions représenter la grille complète par un tableau 2D, utiliser des coordonnées.
- Pour ce cas, nous utiliserons un format de dictionnaire simple comme:

  ```
  {"agent": array([1, 0]), "target": array([0, 3])},
  ```

  où les tableaux représentent les coordonnées x et y.
- Ce choix rend l'observation lisible et facile à déboguer.

Création d'un environnement personnalisé

Environnement __init__

- Nous déclarerons cet espace comme un dictionnaire, les espaces *agent* et *target* étant des boîtes contenant des coordonnées entières.
- Définition de la classe de l'environnement

```
#On crée une classe GridWorldEnv qui hérite de gym.Env,
# ce qui est obligatoire pour un environnement RL compatible Gym.
class GridWorldEnv(gym.Env):
```

- Méthode d'initialisation

```
#L'environnement est un grille carrée de taille size × size,
#par défaut 5 × 5.
def __init__(self, size: int = 5):
```

Création d'un environnement personnalisé

Environnement __init__

- Stockage de la taille de la grille

```
#La taille est enregistrée dans l'objet pour être utilisée plus tard.
self.size = size
```

- Initialisation des positions (agent + cible)

```
#Avant d'appeler reset(), on met des positions non initialisées (-1, -1).
#Cela évite d'avoir de faux états valides dès le départ.
self._agent_location = np.array([-1, -1], dtype=np.int32)
self._target_location = np.array([-1, -1], dtype=np.int32)
```

Création d'un environnement personnalisé

Environnement __init__

- Définition de l'espace d'observation

L'agent observe deux informations :

- Position de l'agent : [x, y] comprise entre 0 et size − 1
- Position de la cible : [x, y] comprise entre 0 et size − 1

```
self.observation_space = gym.spaces.Dict(
    {
        "agent": gym.spaces.Box(0, size - 1, shape=(2,), dtype=int),
        "target": gym.spaces.Box(0, size - 1, shape=(2,), dtype=int),
    }
)
```

Création d'un environnement personnalisé

Environnement __init__

- Définition de l'espace d'observation

  #vgym.spaces.Dict signifie que l'observation est structurée, sous forme d'un dictionnaire Python :

```
{
   "agent": [x, y],
   "target": [x, y]
}
```

- Définition de l'espace d'action

  #L'agent peut faire 4 actions : 0 : aller à droite, 1 : aller en haut, 2 : aller à gauche, 3 : aller en bas

```
self.action_space = gym.spaces.Discrete(4)
```

Création d'un environnement personnalisé

Environnement __init__

- Mapping action → direction

```
self._action_to_direction = {
    0: np.array([1, 0]),   # Move right (positive x)
    1: np.array([0, 1]),   # Move up (positive y)
    2: np.array([-1, 0]),  # Move left (negative x)
    3: np.array([0, -1]),  # Move down (negative y)
}
```

Ce dictionnaire sert de table de correspondance entre le numéro de l'action et le mouvement réel dans la grille.

– Exemple :
  - Action 0, [1, 0] : avancer d'une case vers la droite
  - Action 2, [-1, 0] : avancer d'une case vers la gauche
  - Cela rend le code plus clair que si on utilisait directement des nombres.

Création d’un environnement personnalisé

Construction des observations

- Comme nous devons calculer des observations dans *Env.reset()* et *Env.step()*, il est pratique d'utiliser une méthode auxiliaire *_get_obs* qui convertit l'état interne de l'environnement au format des observations.
- Cela permet de conserver un code concis et facilite la modification ultérieure du format des observations.
- La méthode *_get_obs()* sert à transformer l’état interne de l’environnement en une observation que l’agent pourra utiliser.

Création d'un environnement personnalisé

Construction des observations

- Dans un environnement Gym, l'agent n'accède pas directement aux variables internes.
- Il reçoit une observation (souvent un dictionnaire, un vecteur ou une image).
- Ici, l'état interne est constitué de deux positions :

```
self._agent_location  # position de l'agent dans la grille
self._target_location  # position de la cible à atteindre
```

- La méthode retourne un dictionnaire Python :

```
{
  "agent": self._agent_location,
  "target": self._target_location
}
```

Création d'un environnement personnalisé

Construction des observations

- Ce format correspond exactement à ce qui a été défini dans l'espace d'observation définie dans l'environnement :

```
self.observation_space = gym.spaces.Dict({
   "agent": ...,
   "target": ...
})
```

- Donc l'observation retournée est compatible avec Gym.

Création d'un environnement personnalisé

Construction des observations

- Créer une méthode *_get_obs()* à part apporte plusieurs avantages :
  - Clarté du code
    - Chaque fois qu'on a besoin de l'observation (dans *reset()*, *step()*, etc.),
    - il suffit d'appeler *_get_obs()*.
  - Évolutivité : Plus tard, si on veut ajouter :
    - des obstacles,
    - la distance à la cible,
    - un indicateur de collision,
    - un capteur,

    il suffira de modifier seulement cette fonction.
  - Respect des conventions Gym
    - Chaque environnement possède souvent une fonction similaire (*_get_obs*, observe, etc.).

Création d'un environnement personnalisé

Construction des observations

- Code

```
def _get_obs(self):
    """Convert internal state to observation format.
    Returns:
       dict: Observation with agent and target positions
    """
    return {"agent": self._agent_location, "target": self._target_location}
```

Création d’un environnement personnalisé

Construction des observations

- Nous pouvons également implémenter une méthode similaire pour les informations auxiliaires renvoyées par Env.reset() et Env.step().
- Dans ce cas, nous fournirons la distance de Manhattan entre l'agent et la cible.
- Cela peut être utile pour le débogage et la compréhension de la progression de l'agent.
- Or, ne doit pas être utilisé par l'algorithme d'apprentissage lui-même.

Création d'un environnement personnalisé

Construction des observations

- Code

```
def _get_info(self):
    """Compute auxiliary information for debugging.

    Returns:
        dict: Info with distance between agent and target
    """
    return {
        "distance": np.linalg.norm(
            self._agent_location - self._target_location, ord=1
        )
    }
```

Création d’un environnement personnalisé

Construction des observations

- Parfois, les informations contiennent des données uniquement disponibles dans *Env.step()* (comme les composantes individuelles de la récompense, le succès/l'échec d'une action, etc.).
- Dans ce cas, nous mettons à jour directement le dictionnaire renvoyé par *_get_info* dans la méthode *step*.

Création d’un environnement personnalisé

Fonction de réinitialisation

- La méthode reset() lance un nouvel épisode.
- Elle prend deux paramètres optionnels :
  - Une graine pour une génération aléatoire reproductible,
  - Des options pour une configuration supplémentaire.
- Sur la première ligne, vous devez appeler super().reset(seed=seed) pour initialiser correctement le générateur de nombres aléatoires.
- Dans cette environnement GridWorld, reset() place aléatoirement l'agent et la cible sur la grille, en veillant à ce qu'ils ne commencent pas au même endroit.
- Nous retournons l'observation initiale et les informations sous forme de tuple.

Création d'un environnement personnalisé

Fonction de réinitialisation

- Initialisation correcte

```
super().reset(seed=seed)
```

C'est très important :

- Gymnasium utilise une gestion interne de la graine aléatoire (np_random),
- donc cet appel initialise le générateur de nombres aléatoires pour l'environnement.
- Sans cet appel, l'environnement ne serait pas reproductible.

Création d'un environnement personnalisé

Fonction de réinitialisation

- Placement aléatoire de l'agent

```
self._agent_location = self.np_random.integers(0, self.size, size=2, dtype=int)
```

L'agent est placé aléatoirement dans la grille.

self.np_random : le générateur aléatoire interne fourni par Gym.

size=2 : coordonnées (x, y).

Cela donne un tableau comme : [2, 4] # ex : agent en x=2, y=4

Création d'un environnement personnalisé

Fonction de réinitialisation

- Placement aléatoire de la cible (différente de l'agent)

```
self._target_location = self._agent_location
while np.array_equal(self._target_location, self._agent_location):
    self._target_location = self.np_random.integers( 0, self.size, size=2, dtype=int)
```

  - On commence par assigner la cible au même endroit que l'agent.
  - Puis, on entre dans une boucle while qui continue tant que les deux positions sont identiques.
  - À chaque boucle, on tire une nouvelle position de cible.
  - But :
    - garantir que la cible n'apparaît jamais sur la position de l'agent.
    - C'est une technique simple et efficace pour éviter une collision initiale.

Création d'un environnement personnalisé

Fonction de réinitialisation

- Création de l'observation et du dictionnaire info

```
observation = self._get_obs()
info = self._get_info()
```

_get_obs() : renvoie un dict contenant : {"agent": ..., "target": ...}

_get_info() : fournit des informations supplémentaires (statistiques, distances, etc.)

- Retour de la fonction

```
return observation, info
```

  - Le reset retourne exactement ce que Gymnasium exige :
    - observation initiale
    - info (métadonnées)

Création d’un environnement personnalisé

Fonction étape

- La méthode *step()* contient la logique principale de l'environnement.
- Elle prend une action en entrée, met à jour l'état de l'environnement et renvoie le résultat.
- C'est ici que sont gérées la physique, les règles du jeu et le système de récompenses.

Création d’un environnement personnalisé

Fonction étape

- Pour GridWorld, nous devons :
  1. Convertir l'action discrète en une direction de déplacement ;
  2. Mettre à jour la position de l'agent (en vérifiant les limites de l'environnement) ;
  3. Calculer la récompense selon que la cible a été atteinte ou non ;
  4. Déterminer si l'épisode doit se terminer ;
  5. Renvoyer toutes les informations nécessaires.

Création d'un environnement personnalisé

Fonction étape

- Mapping action vers mouvement

  #L'agent choisit une action entre 0 et 3.

  direction = self._action_to_direction[action]

  Cette ligne transforme l'action en un vecteur de déplacement, grâce au dictionnaire :

  {

  0: [1, 0] : aller à droite

  1: [0, 1] : aller en haut

  2: [-1, 0] : aller à gauche

  3: [0, -1] : aller en bas

  }

Création d’un environnement personnalisé

Fonction étape

- Mise à jour de la position de l’agent

```
#On ajoute le vecteur direction à la position actuelle.
self._agent_location = np.clip(
    self._agent_location + direction, 0, self.size - 1
)
```

np.clip() empêche l’agent de sortir de la grille :

- minimum autorisé : 0
- maximum : size - 1

– Cela évite que l’agent "tombe" en dehors du monde.

Création d'un environnement personnalisé

Fonction étape

- Vérifier si l'agent a atteint la cible

  #Si la nouvelle position de l'agent = position de la cible alors l'épisode terminé.

  terminated = np.array_equal(self._agent_location, self._target_location)

  – Cette variable indique à l'algorithme qu'il doit recommencer un nouvel épisode.

- Indiquer si l'épisode est interrompu

  truncated = False

  – Ici, on ne limite pas le nombre d'actions.

  – Donc il n'y a pas de troncation

Création d'un environnement personnalisé

Fonction étape

- Calcul de la récompense

  reward = 1 if terminated else 0

  - Si l'agent atteint la cible alors la récompense = +1
  - Sinon, la récompense = 0
  - C'est une structure de récompense très simple.
- Préparer l'observation et les infos de debug

  observation = self._get_obs()

  info = self._get_info()

  _get_obs() renvoie les positions sous forme d'un dictionnaire {agent: ..., target: ...}

  _get_info() peut contenir des informations supplémentaires.

Création d'un environnement personnalisé

Fonction étape

- Retourner le résultat de l'étape

  return observation, reward, terminated, truncated, info

  – Cette structure est obligatoire pour les environnements Gymnasium :
    - observation : nouvel état
    - reward : récompense obtenue
    - terminated : fin naturelle de l'épisode ou pas encore?
    - truncated : interruption forcée ou non ?
    - info : métadonnées (debug, distances, etc.)

Erreurs fréquentes de la modélisation d'environnements

Erreurs fréquentes de la modélisation d'environnements

- Jusqu’à présent, nous avons vu la structure de base de modélisation d’un environnement.
- Or, lors de la modélisation, des erreurs fréquentes peuvent être commises par un débutante comme :
  - Problèmes liés à la conception des récompenses
  - Problèmes de représentation de l'état
  - Problèmes liés à l'espace d'actions
  - Erreurs de gestion des limites

Erreurs fréquentes de la modélisation d'environnements

- Problèmes liés à la conception des récompenses
  - Problème : Récompenser uniquement à la fin (récompenses rares)

```
# This makes learning very difficult!
reward = 1 if terminated else 0
```

  - Solution : Fournir un retour d'information intermédiaire

```
# Option 1: Small step penalty to encourage efficiency
reward = 1 if terminated else -0.01
# Option 2: Distance-based reward shaping
distance = np.linalg.norm(self._agent_location - self._target_location)
reward = 1 if terminated else -0.1 * distance
```

  - Explication
    - Option 1 : Pénalité légère à chaque pas : reward = 1 if terminated else -0.01
    - Si l'agent atteint la cible (terminated = True), il reçoit : +1
    - Sinon, à chaque pas il reçoit une petite pénalité : –0.01
    - Cela encourage l'agent à atteindre le but rapidement en évitant de tourner inutilement dans la grille.

Erreurs fréquentes de la modélisation d'environnements

- Problèmes liés à la conception des récompenses
  - Explication
    - Option 2 : Récompense basée sur la distance
    - distance = np.linalg.norm(self._agent_location - self._target_location)
    - reward = 1 if terminated else -0.1 * distance
    - On calcule la distance euclidienne entre l'agent et la cible :

      distance = sqrt($(x1 - x2)^2 + (y1 - y2)^2$)
    - La récompense devient : +1 si l'agent atteint la cible, –0.1 × distance sinon
    - Plus l'agent s'éloigne, plus la pénalité est élevée.
    - Plus il se rapproche, plus la pénalité est faible.
    - Cela crée un gradient de récompense qui guide l'agent vers la cible.
    - L'agent comprend dans quelle direction aller. L'apprentissage devient plus rapide et plus stable.
    - Le problème de récompenses rares est réduit.

Erreurs fréquentes de la modélisation d'environnements

- Problèmes de représentation de l'état
  - Problème : Inclusion d'informations non pertinentes ou omission de détails cruciaux.

```
# Too much info - agent doesn't need grid size in every observation
obs = {"agent": self._agent_location, "target": self._target_location, "size": self.size}
# Too little info - agent can't distinguish different positions
obs = {"distance": distance}  # Missing actual positions!
```

  - Solution : Inclure précisément ce qui est nécessaire pour prendre des décisions optimales.

```
# Just right - positions are sufficient for navigation
obs = {"agent": self._agent_location, "target": self._target_location}
```

Erreurs fréquentes de la modélisation d'environnements

- Problèmes de représentation de l'état
  - Explication
    - L’observation est un dictionnaire contenant deux éléments :
    - "agent" : la position actuelle de l’agent dans la grille (exemple : [2, 4])
    - "target" : la position de la cible (exemple : [0, 1])
    - Cet ensemble permet à l’agent de savoir :
      - où il se trouve
      - où il doit aller
    - Donc, il peut calculer une stratégie de navigation.
    - Dans un GridWorld simple :
      - il n’y a pas d’obstacles
      - l’environnement est une grille régulière
      - les actions sont simples (haut, bas, gauche, droite)
    - Alors l’information minimale nécessaire pour décider de la prochaine action est simplement les deux positions.
    - Pas besoin d’une image, d’un vecteur plus long, ou de données complexes.

Erreurs fréquentes de la modélisation d'environnements

- Problèmes liés à l'espace d'actions
  - Problème : Actions incohérentes ou impossibles à exécuter
    - # Bad: Agent can move diagonally but environment doesn't support it
    - self.action_space = gym.spaces.Discrete(8) # 8 directions including diagonals

    - # Bad: Continuous actions for discrete movement
    - self.action_space = gym.spaces.Box(-1, 1, shape=(2,)) # Continuous x,y movement
  - Explication :
    - Cas 1 :
      - Ici, on définit un espace d'actions avec 8 actions :
      - 4 directions classiques (haut, bas, gauche, droite) + 4 diagonales (haut-droite, haut-gauche, bas-droite, bas-gauche).
      - Problème : L'environnement GridWorld ne prévoit que 4 actions valides (pas de déplacement diagonal).
      - Donc, l'agent pourrait choisir une action que l'environnement ne sait pas gérer.
      - Cela crée une incohérence entre ce que l'agent peut faire et ce que l'environnement accepte.

Erreurs fréquentes de la modélisation d'environnements

- Problèmes liés à l'espace d'actions
  - Explication :
    - Cas 2 :
      - Ici, les actions sont continues : l’agent peut envoyer un vecteur (x, y) avec n’importe quelle valeur entre –1 et +1
      - cela impliquerait des mouvements continus (comme dans un drone ou une voiture)
      - Problème : Le GridWorld fonctionne avec mouvements discrets (cases de la grille).
      - Il n’accepte pas des déplacements comme : (0.25, -0.78)
      - L’environnement ne peut pas convertir correctement une action continue en déplacement discret.
      - Cela rend le comportement imprévisible ou entraîne des erreurs.

Erreurs fréquentes de la modélisation d'environnements

- Erreurs de gestion des limites
  - Problème : Autorisation d'états invalides ou comportement ambigu aux limites

```
# Bad: Agent can go outside the grid
self._agent_location = self._agent_location + direction  # No bounds checking!


# Unclear: What happens when agent hits wall?
if np.any(self._agent_location < 0) or np.any(self._agent_location >= self.size):
# Do nothing? Reset episode? Give penalty? Unclear!
```

Erreurs fréquentes de la modélisation d'environnements

- Erreurs de gestion des limites
  - Solution : Gestion claire et cohérente des limites

```
# Clear: Agent stays in place when hitting boundaries
self._agent_location = np.clip(
    self._agent_location + direction, 0, self.size - 1
)
```

  - Explication :
    - Ce code met à jour la position de l'agent tout en l'empêchant de sortir de la grille.
    - Calcul du nouveau déplacement
      - self._agent_location + direction
      - self._agent_location : position actuelle de l'agent, par ex. [2, 4]
      - direction : mouvement choisi (ex : [1, 0] → aller à droite)
      - Cela donne la prochaine position.

Erreurs fréquentes de la modélisation d'environnements

- Erreurs de gestion des limites
  - Solution : Gestion claire et cohérente des limites
    - Empêcher l'agent de sortir de la grille

      np.clip(..., 0, self.size - 1)
      - np.clip impose une valeur minimale et maximale. valeur min = 0, valeur max = self.size - 1 (dernière case de la grille)
      - Si la grille est de taille 5 (indices 0 à 4) et que l'agent essaie d'aller : à droite depuis [4, 2], nouvelle position calculée = [5, 2] (hors limites). np.clip corrige automatiquement en : [4, 2]. L'agent reste dans la grille.
      - Parce que cette méthode :
        - » évite d'écrire plusieurs conditions if pour vérifier les limites ;
        - » gère les frontières de manière élégante ;
        - » garantit que la position ne dépasse jamais les bornes.
      - C'est une façon propre, compacte et lisible de maintenir l'agent dans un espace discret limité.

Enregistrement et création de l'environnement

Enregistrement et création de l'environnement

- Avant d’utiliser votre environnement personnalisé, il est plus pratique de l'enregistrer auprès de Gymnasium afin de pouvoir le créer avec gymnasium.make(), comme les environnements intégrés.
- L'identifiant de l'environnement comporte trois éléments :
  - un espace de noms facultatif (ici : gymnasium_env),
  - un nom obligatoire (ici : GridWorld)
  - une version facultative mais recommandée (ici : v0).
- Vous pouvez l'enregistrer sous GridWorld-v0, GridWorld ou gymnasium_env/GridWorld, mais le format complet est recommandé pour plus de clarté.

Enregistrement et création de l'environnement

- Le code d'enregistrement

```
# Register the environment so we can create it with gym.make()
gym.register(
   id="gymnasium_env/GridWorld-v0",
   entry_point=GridWorldEnv,
   max_episode_steps=300,  # Prevent infinite episodes
)
```

- Explication
  - Ce code sert à enregistrer votre environnement personnalisé dans Gym. Cela permet ensuite de le créer facilement avec :

```
env = gym.make("gymnasium_env/GridWorld-v0")
```

  - Sans cet enregistrement, Gym ne connaît pas votre environnement, donc gym.make() ne fonctionnerait pas.

Enregistrement et création de l'environnement

- Le code d'enregistrement
  - Explication
    - L'identifiant unique de l'environnement.

      id="gymnasium_env/GridWorld-vo"
    - nom du package ou groupe

      "gymnasium_env"
    - nom + version (v0, v1, …) Il doit être unique pour éviter les conflits.

      "GridWorld-vo"
    - entry_point indique quelle classe Python doit être instanciée lorsque Gym crée l'environnement.

      entry_point=GridWorldEnv

Enregistrement et création de l'environnement

- Le code d’enregistrement
- Explication
  - C’est votre classe personnalisée qui contient : *reset(), step(), render() action_space* etc.
  - Ce paramètre impose un nombre maximum d’étapes par épisode.

    max_episode_steps=300
  - C’est dans le but d’éviter :
    - les épisodes qui durent infiniment
    - les agents qui tournent en rond
    - les blocages en apprentissage
  - Après 30 actions, Gym arrête automatiquement l’épisode en mettant :
  - truncated = True

Enregistrement et création de l'environnement

- Une fois inscrit, vous pouvez consulter tous les environnements disponibles avec gymnasium.pprint_registry() et créer des instances avec gymnasium.make().

```
import gymnasium as gym
# Create the environment like any built-in environment
env = gym.make("gymnasium_env/GridWorld-v0")
print("our environement :", env)
# Customize environment parameters
env = gym.make("gymnasium_env/GridWorld-v0", size=10)
print("Customize environment parameters : ", env.unwrapped.size)
# Create multiple environments for parallel training
vec_env = gym.make_vec("gymnasium_env/GridWorld-v0",
num_envs=3)
print("our multiple environement :", vec_env)
```

Débogage de l’environnement

Débogage de l'environnement

- Si votre environnement ne fonctionne pas comme prévu, voici quelques stratégies de débogage courantes :
  - Vérifier la validité de l'environnement

```python
from gymnasium.utils.env_checker import check_env
# This will catch many common issues
try:
    check_env(env)
    print("Environment passes all checks!")
except Exception as e:
    print(f"Environment has issues: {e}")
```

```
Environment passes all checks!
/usr/local/lib/python3.12/dist-packages/gymnasium/utils/e
  logger.warn(
```

Débogage de l'environnement

- Tests manuels avec des actions connues

```python
# Test specific action sequences to verify behavior
env = gym.make("gymnasium_env/GridWorld-v0")
obs, info = env.reset(seed=42)  # Use seed for reproducible testing
print(f"Starting position - Agent: {obs['agent']}, Target: {obs['target']}")
# Test each action type
actions = [0, 1, 2, 3]  # right, up, left, down
for action in actions:
    old_pos = obs['agent'].copy()
    obs, reward, terminated, truncated, info = env.step(action)
    new_pos = obs['agent']
    print(f"Action {action}: {old_pos} -> {new_pos}, reward={reward}")
```

```
Starting position - Agent: [0 3], Target: [3 2]
Action 0: [0 3] -> [1 3], reward=0
Action 1: [1 3] -> [1 4], reward=0
Action 2: [1 4] -> [0 4], reward=0
Action 3: [0 4] -> [0 3], reward=0
```

Débogage de l'environnement

- Problèmes de débogage courants

```
# Problème n° 1 : J'ai oublié d'appeler super().reset()
def reset(self, seed=None, options=None):
    # super().reset(seed=seed)  # Missing this line
    # Results in: possibly incorrect seeding

# Problème 2 : Association d'actions incorrecte
self._action_to_direction = {
    0: np.array([1, 0]),   # right
    # up - Mais est-ce vraiment « en haut » dans votre système de coordonnées ?
    1: np.array([0, 1]),
    2: np.array([-1, 0]),  # left
    3: np.array([0, -1]),  # down
}
# Problème 3 : Gestion incorrecte des limites
# Cela permet à l'agent de sortir de la grille !
self._agent_location = self._agent_location + direction  #  No bounds checking
```

Utilisation des wrappers

Utilisation des wrappers

- Il arrive que vous souhaitiez modifier le comportement de votre environnement sans toucher à son implémentation principale.
- Les wrappers sont la solution idéale :
  - ils vous permettent d'ajouter des fonctionnalités telles que la modification des formats d'observation, l'ajout de limites de temps ou la modification des récompenses,
  - sans toucher au code source de votre environnement.

Utilisation des wrappers

- Exemple : Aplatit l'observation en un simple tableau.

```
1   from gymnasium.wrappers import FlattenObservation
2   # Original observation is a dictionary
3   env = gym.make('gymnasium_env/GridWorld-v0')
4   print(" Original observation : ", env.observation_space)
5   obs, info = env.reset()
6   print(" obs : ", obs)
7   # Wrap it to flatten observations into a single array
8   wrapped_env = FlattenObservation(env)
9   print("Wrap it to flatten observations into a single array",wrapped_env.observation_space)
10  obs, info = wrapped_env.reset()
11  print(" flatten  observation : ", obs)
```

```
 Original observation :  Dict('agent': Box(0, 4, (2,), int64), 'target': Box(0, 4, (2,), int64))
 obs :  {'agent': array([2, 2]), 'target': array([0, 3])}
Wrap it to flatten observations into a single array Box(0, 4, (4,), int64)
 flatten  observation :  [2 4 0 4]
```

Utilisation des wrappers

- Remarque : Ceci est particulièrement utile lors de l'utilisation d'algorithmes qui attendent des formats d'entrée spécifiques (comme les réseaux neuronaux qui nécessitent des tableaux 1D au lieu de dictionnaires).

Fonctionnalités avancées de l'environnement

Fonctionnalités avancées de l'environnement

- Une fois les fonctions de base opérationnelles, vous pouvez ajouter des fonctionnalités plus sophistiquées :
  - Ajout du rendu.

```python
def render(self):
    """Render the environment for human viewing."""
    if self.render_mode == "human":
        # Print a simple ASCII representation
        for y in range(self.size - 1, -1, -1):  # Top to bottom
            row = ""
            for x in range(self.size):
                if np.array_equal([x, y], self._agent_location):
                    row += "A "  # Agent
                elif np.array_equal([x, y], self._target_location):
                    row += "T "  # Target
                else:
                    row += ". "  # Empty
            print(row)
        print()
```

Fonctionnalités avancées de l'environnement

- Environnements paramétrés

```python
def __init__(self, size: int = 5, reward_scale: float = 1.0, step_penalty: float = 0.0):
    self.size = size
    self.reward_scale = reward_scale
    self.step_penalty = step_penalty
    # ... rest of init ...

def step(self, action):
    # ... movement logic ...

    # Flexible reward calculation
    if terminated:
        reward = self.reward_scale  # Success reward
    else:
        reward = -self.step_penalty  # Step penalty (0 by default)
```

Conseils pour la conception d'environnements réalistes

Conseils pour la conception d'environnements réalistes

- Commencez par la simplicité, complexifiez progressivement
  - Premièrement : assurez-vous que les mouvements de base et l'atteinte des objectifs fonctionnent.
  - Ensuite : ajoutez des obstacles, des objectifs multiples ou une contrainte de temps.
  - Enfin : intégrez des dynamiques complexes, une observabilité partielle ou des interactions multi-agents.

Conseils pour la conception d'environnements réalistes

- Conception favorisant l'apprentissage
  - Critères de réussite clairs : l'agent doit savoir quand il réussit.
  - Difficulté raisonnable : ni trop facile (trivial), ni trop difficile (impossible).
  - Règles cohérentes : une même action dans un même état doit avoir le même effet.
  - Observations pertinentes : incluez toutes les informations nécessaires à une prise de décision optimale.

Conseils pour la conception d'environnements réalistes

- Réfléchissez à votre question de recherche.
  - Navigation : concentrez-vous sur le raisonnement spatial et la planification de trajectoires.
  - Contrôle : privilégiez la dynamique, la stabilité et la continuité des actions.
  - Stratégie : intégrez des informations partielles, la modélisation des adversaires ou la planification à long terme.
  - Optimisation : définissez des compromis clairs et des contraintes de ressources.

conclusion

Chapter 3 : Advanced Reinforcement Learning

Deep Reinforcement Learning

Introduction

- Le Deep Reinforcement Learning (DRL) est considéré comme une extension de l'apprentissage par renforcement classique qui utilise des réseaux de neurones profonds.
- C'est dans le but d'approximer :
  - Une fonction de valeur V(s),
  - Une fonction action-valeur Q(s,a),
  - Ou directement une politique $\pi(a|s)$.
- Ce type de réseau est particulièrement adapté aux environnements :
  - À grands espaces d'états (images, signaux, séries temporelles),
  - Continus,
  - Partiellement observables.

Introduction

- Un agent interagit avec un environnement selon le cycle suivant :
  - Observation de l'état s_t
  - Choix d'une action a_t via une politique
  - Réception d'une récompense r_t
  - Transition vers un nouvel état s_{t+1}
  - Mise à jour du modèle (réseau de neurones)
- L'objectif est de maximiser la récompense cumulée actualisée.

Deep Q-Learning

Deep Q-Learning

- Dans le cas Q-learning, la fonction de valeur d'action Q(s,a) est stockée sous forme d'une table Q.
- Il est associe explicitement à chaque paire (état, action) une valeur numérique.
- Le Q-learning apprend une fonction :

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \, a' \max Q(s',a') - Q(s,a)]$$

- Cette approche fonctionne bien lorsque l'espace d'états et d'actions est petit et discret.
- Or, elle devient rapidement inapplicable pour des systèmes complexes dont le nombre d'états possibles est très élevé, voire continu (voir tutoriel 2).

Deep Q-Learning

- Dans ce contexte, Le Deep Q-Network (DQN) résout cette limitation en remplaçant la table Q par un réseau de neurones artificiels.
- Ce réseau prend en entrée une représentation de l'état.
- Il produit en sortie les valeurs Q(s,a) pour toutes les actions possibles.
- Grâce à sa capacité de généralisation, le réseau RNA peut estimer des valeurs Q pour des états jamais rencontrés auparavant.
- Cela permet d'appliquer le Q-learning à des problèmes de grande dimension.

Deep Q-Learning

- L'apprentissage consiste à ajuster les poids du réseau RNA afin d'approximer au mieux la fonction $Q(s,a;\theta)$ optimale, à partir des expériences accumulées par l'agent.
- Innovations clés dans ce cas sont :
  - Experience Replay
  - Target Network
  - Exploration ε-greedy

Concepts clés du Deep Q-Learning

Concepts clés du Deep Q-Learning

- Approximation de la fonction Q :
  - Au lieu d'utiliser une table pour stocker les valeurs Q de chaque paire état-action, le DQN utilise un réseau de neurones pour approximer ces valeurs.
  - L'entrée du réseau est l'état, et la sortie est un ensemble de valeurs Q pour toutes les actions possibles.
- Relecture d'expérience :
  - Pour stabiliser l'entraînement, le DQN utilise une mémoire tampon (mémoire de relecture) pour stocker les expériences (état, action, récompense, état suivant).
  - Le réseau est entraîné sur des mini-lots aléatoires d'expériences provenant de cette mémoire tampon, ce qui rompt la corrélation entre les expériences consécutives et améliore l'efficacité de l'échantillonnage.

Concepts clés du Deep Q-Learning

- Réseau cible :
  - Le DQN introduit un second réseau de neurones, appelé réseau cible, qui sert à calculer les valeurs Q cibles.
  - Ce réseau cible est mis à jour moins fréquemment que le réseau principal afin d'éviter les oscillations rapides pendant l'apprentissage.
- Équation de Bellman dans le DQN :
  - La règle de mise à jour du DQN est basée sur l'équation de Bellman, comme pour le Q-Learning.

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a'; \theta^{-}) - Q(s,a; \theta)]$$

  - Où :
    - θ représente les poids du réseau Q principal,
    - θ^{-} les poids du réseau Q cible,

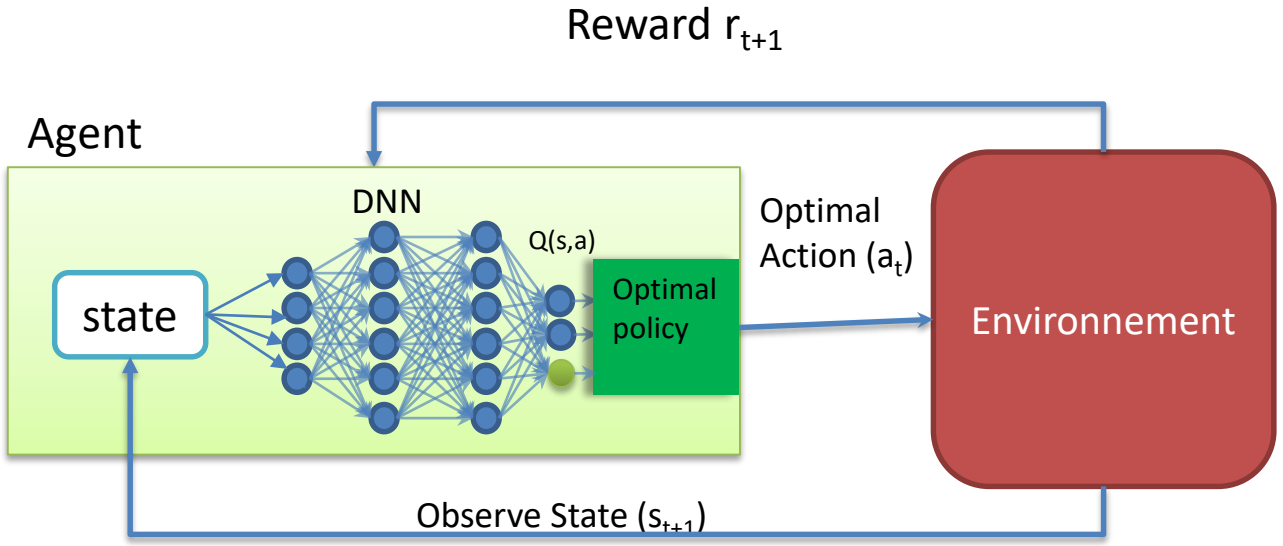
Architecture de DQN

Architecture de DQN

- Une architecture de DQN repose sur l'utilisation d'un réseau de neurones profond chargé d'approximer la fonction de valeur d'action Q(s,a).
- L'état de l'environnement est fourni en entrée du réseau profond (un vecteur de capteurs, une image, …).
- Puis transformé à travers plusieurs couches cachées:
  - Convolutionnelles pour des entrées visuelles,
  - Entièrement connectées pour des états vectoriels,
- La couche de sortie comporte autant de neurones que d'actions possibles.
- Chaque neurone représente la valeur Q associée à une action donnée.

Architecture de DQN

- Cette architecture permet à l'agent de sélectionner l'action ayant la valeur Q maximale, tout en généralisant efficacement sur de grands espaces d'états.
- Cela rend le DQN adapté aux problèmes complexes de prise de décision.



Architecture de DQN

Réseau de neurones

- Le réseau approxime la fonction de valeur Q :

  $Q(s, a, \theta)$,

  où θ représente les paramètres entraînables.

- Par exemple, dans les jeux Atari:
  - L'entrée peut être constituée des pixels bruts de l'écran de jeu.
  - La sortie est un vecteur de valeurs Q correspondant à chaque action possible.

Architecture de DQN

Relecture d'expérience

- Pour stabiliser l'entraînement, les réseaux DQN stockent les expériences passées (s, a, r, s′) dans une mémoire tampon de relecture.
- Pendant l'entraînement, des mini-lots d'expériences sont échantillonnés aléatoirement dans cette mémoire tampon.
- Cela rompt la corrélation entre les expériences consécutives et améliore la généralisation.

Architecture de DQN

Réseau cible

- Un réseau cible distinct, de paramètres θ^-, est utilisé pour calculer les valeurs Q cibles lors des mises à jour.
- Le réseau cible est mis à jour périodiquement avec les poids du réseau principal afin d'assurer sa stabilité.

Fonction de perte

- La fonction de perte mesure la différence entre les valeurs Q prédites et les valeurs Q cibles.

$$L(\theta) = E[(r+\gamma \max_{a'} Q(s',a'; \theta^-) - Q(s,a; \theta))^2]$$

Architecture de DQN : Réseau cible

Architecture de DQN : Réseau cible

- Dans un DQN classique, si le même réseau de neurones est utilisé à la fois pour :
  - Estimer la valeur Q actuelle $q(s,a;\theta)$,
  - Calculer la cible d'apprentissage $y=r+\gamma \max_a Q(s',a;\theta)$
- Cela crée un problème majeur :
  - la cible dépend des paramètres que l'on essaie justement d'apprendre.
- Résultat :
  - La cible change constamment,
  - L'apprentissage devient instable,
  - Les oscillations ou la divergence peuvent apparaître.

Architecture de DQN : Réseau cible

- Pour résoudre ce problème, le DQN introduit un deuxième réseau, appelé réseau cible, noté généralement Q_{target}
- Le réseau principal (online) est entraîné à chaque itération.
- Le réseau cible est figé temporairement et mis à jour seulement toutes les N itérations.
- La mise à jour se fait par copie :$\theta^{-} \leftarrow \theta$
- Grâce au réseau cible :
  - la valeur cible devient plus stable dans le temps,
  - les mises à jour sont moins bruitées,
  - la convergence est plus fiable.

Architecture de DQN : Réseau cible

- La cible devient alors : $y = r + \gamma \max_a Q_{target}(s', a; \theta)$
- Ce découplage réduit le phénomène de feedback instable, où le réseau apprend à partir de ses propres prédictions erronées.
- Les bénéfices concrets sont :
  - Apprentissage plus stable
  - Réduction des oscillations et divergences
  - Meilleure convergence
  - Amélioration des performances globales

Le processus d’apprentissage

Le processus d’apprentissage

- Le processus d’apprentissage d'un DQN se constitue des étapes suivantes :
  1. **Initialisation**
     - tampon de relecture,
     - du réseau principal (θ)
     - du réseau cible (θ^-).
     - Choix des hyperparamètres :
       - taux d'apprentissage (α),
       - le facteur d'actualisation (γ)
       - le taux d'exploration (ϵ).
  2. **Exploration vs. Exploitation :**
     - Utilisation d'une politique ϵ-gloutonne pour équilibrer exploration et exploitation :
       - Avec une probabilité ϵ, sélection d'une action aléatoire à explorer.
       - Sinon, choix de l'action ayant la valeur Q la plus élevée selon le réseau actuel.

Le processus d’apprentissage

3. **Acquisition d'expérience :**
   - Interaction avec l'environnement, acquisition d'expériences (s, a, r, s′) et stockage dans le tampon de relecture.
4. **Mises à jour de l'entraînement :**
   - Échantillonner un mini-lot d'expériences à partir du tampon de relecture.
   - Calculer les valeurs Q cibles à l'aide du réseau cible.
   - Mettre à jour le réseau principal en minimisant la fonction de perte par descente de gradient.
5. **Mise à jour du réseau cible :**
   - Copier périodiquement les poids du réseau principal vers le réseau cible pour assurer sa stabilité.
6. **Décroissance du taux d'exploration :**
   - Diminuer progressivement ϵ au fil du temps pour passer de l'exploration à l'exploitation.

pseudo-code du DQN

pseudo-code du DQN

- 1. Début de l'épisode.
- 2. Sélection et exécution des actions dans un agent d'apprentissage par renforcement
  - Exécution de l'action a_t à partir de l'état s_t. Initialement, l'action est sélectionnée aléatoirement.
  - observation de l'état suivant s_{t+1}
  - Au fil du temps, l'action prédite par le réseau est utilisée selon une politique epsilon-greedy.
  - Cette politique privilégie l'action ayant la valeur Q la plus élevée.
  - Dans un réseau de neurones classique, la prédiction la plus probable est sélectionnée.
- 3. Calcul de la récompense associée à l'action.

pseudo-code du DQN

- 4. stockage des données
  - Stockage de l'état actuel, de l'action, de la récompense et de l'état suivant dans le tampon de relecture.
  - Ceci permet l'entraînement par lots du réseau de neurones.
- 5. répétition des étapes
  - s_{t+1} correspond au nouvel état s_t.
  - Répétition des étapes 2 à 4 jusqu'à ce que la taille du lot soit atteinte.

pseudo-code du DQN

- 6. Entraînement par lots et mise à jour du réseau cible dans un DQN
  - Lancement de l'entraînement par lots du réseau de neurones une fois la taille du lot atteinte.
  - Veuillez noter que la représentation de la fonction de perte diffère selon l'implémentation.
  - Après avoir atteint N itérations, les poids du réseau de prédiction sont copiés dans le réseau cible.
- 7. Fin de l'épisode.
- 8. Répétez les étapes 1 à 7 jusqu'à l'obtention de la valeur Q optimale.

Implémentation de l'apprentissage

Implémentation de l'apprentissage

- Implémentation de l'apprentissage par renforcement profond (DQN) via python.
- Nous allons implémenter une version simple de l'apprentissage par renforcement profond (DQN) en utilisant l'environnement frozen lake d'OpenAI Gym.

1. Initialisation de l'environnement

```
env = gym.make("FrozenLake-v1", is_slippery=True, render_mode='rgb_array')
n_states = env.observation_space.n   # 16 états
n_actions = env.action_space.n       # 4 actions (Left, Down, Right, Up)
```

   - Explication
     - Ce code initialise l'environnement FrozenLake-v1 de la bibliothèque Gym.

Implémentation de l'apprentissage

- Explication
  - Le paramètre `is_slippery=True` indique que l'environnement est stochastique :
    - Lorsque l'agent choisit une action,
    - Il peut glisser et être déplacé vers une direction non souhaitée,
    - Ce qui rend les transitions entre états probabilistes et le problème plus réaliste et plus difficile.
  - L'option `render_mode='rgb_array'` permet de générer une représentation visuelle de l'environnement sous forme d'images, utile pour l'affichage ou l'enregistrement de vidéos.

Implémentation de l'apprentissage

2. Encodage de l’état

```
def one_hot(state, n_states):
    vec = np.zeros(n_states)
    vec[state] = 1.0
    return vec
```

– Explication

- Définit une fonction d’encodage one-hot pour un état discret.
- Prend en entrée l’état courant state et le nombre total d’états n_states.
- Crée un vecteur vec de taille n_states initialisé à zéro.
- Place la valeur 1.0 à l’indice correspondant à l’état courant.
- Garantit une représentation unique et non ambiguë de chaque état.
- Permet d’utiliser des états discrets comme entrée numérique d’un Deep Q-Learning.

Implémentation de l'apprentissage

3. Réseau de neurones DQN

```python
class DQN(nn.Module):

    def __init__(self, input_dim, output_dim):
        super(DQN, self).__init__()

        self.fc1 = nn.Linear(input_dim, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, output_dim)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)
```

Implémentation de l'apprentissage

3. Réseau de neurones DQN
   - Explication
     - Définit une classe DQN qui hérite de nn.Module, représentant un réseau de neurones pour le Deep Q-Learning.
     - Le constructeur __init__ initialise l'architecture du réseau à partir de la dimension d'entrée input_dim (état encodé) et de la dimension de sortie output_dim (nombre d'actions).
     - La couche fc1 est une couche entièrement connectée qui projette l'entrée vers 64 neurones cachés.
     - La couche fc2 est une seconde couche cachée de 64 neurones, permettant d'apprendre des représentations plus complexes.
     - La couche fc3 est la couche de sortie qui produit une valeur Q pour chaque action possible.
     - La méthode forward définit la propagation avant du réseau.
     - La fonction d'activation ReLU est appliquée après les deux premières couches pour introduire de la non-linéarité.
     - La sortie finale correspond aux Q-values estimées pour toutes les actions dans un état donné.

Implémentation de l'apprentissage

4. les Hyperparamètres

```
learning_rate = 0.001
gamma = 0.99
epsilon = 1.0
epsilon_min = 0.05
epsilon_decay = 0.995
batch_size = 64
memory_size = 10000
target_update_freq = 500
episodes = 5000
```

– Explication

- learning_rate fixe le pas d'apprentissage de l'optimiseur lors de la mise à jour des poids du réseau.
- gamma représente le facteur de discount, qui contrôle l'importance accordée aux récompenses futures.
- epsilon initialise le taux d'exploration dans la stratégie ε-greedy.

Implémentation de l'apprentissage

4. les Hyperparamètres
   - Explication
     - epsilon_min définit la valeur minimale d'exploration afin d'éviter une exploitation pure trop précoce.
     - epsilon_decay contrôle la décroissance progressive de epsilon au fil des épisodes.
     - batch_size correspond au nombre d'expériences utilisées à chaque étape d'apprentissage par mini-lot.
     - memory_size fixe la capacité maximale du replay buffer pour stocker les expériences passées.
     - target_update_freq indique la fréquence de mise à jour du réseau cible, exprimée en nombre d'itérations.
     - episodes définit le nombre total d'épisodes d'entraînement de l'agent.

Implémentation de l'apprentissage

5. Initialisation des réseaux

```
policy_net = DQN(n_states, n_actions)
target_net = DQN(n_states, n_actions)

# Le target network démarre avec les mêmes poids
target_net.load_state_dict(policy_net.state_dict())
target_net.eval()

optimizer = optim.Adam(policy_net.parameters(), lr=learning_rate)

# Mémoire de rejouement (Experience Replay)
memory = deque(maxlen=memory_size)
```

– Explication

- Initialise le réseau principal policy_net, utilisé pour estimer les Q-values et sélectionner les actions.
- Initialise le réseau cible target_net, ayant la même architecture que le réseau principal.

Implémentation de l'apprentissage

5. Initialisation des réseaux
   - Explication
     - Copie les poids du réseau principal vers le réseau cible afin de démarrer avec des paramètres identiques.
     - Met le réseau cible en mode évaluation avec `eval()`, car il n'est pas entraîné directement.
     - Définit l'optimiseur `Adam` appliqué aux paramètres du réseau principal, avec le taux d'apprentissage spécifié.
     - Crée une mémoire `memory` de rejouement (Experience Replay) à l'aide d'une file `deque`.
     - Limite la taille de la mémoire à `memory_size` pour stocker les transitions passées sous la forme (état, action, récompense, état suivant, terminal).

Implémentation de l'apprentissage

6. Politique ε-greedy

```
def select_action(state, epsilon):
    if random.random() < epsilon:
        return env.action_space.sample()
    else:
        state_tensor = torch.FloatTensor(state).unsqueeze(0)
        q_values = policy_net(state_tensor)
        return torch.argmax(q_values).item()
```

– Explication

- Définit une fonction de sélection d'action basée sur la stratégie ε-greedy.
- Compare un nombre aléatoire à epsilon pour décider entre exploration et exploitation.
- Si l'exploration est choisie, sélectionne une action aléatoire à partir de l'espace d'actions de l'environnement.

Implémentation de l'apprentissage

6. Politique ε-greedy
   - Explication
     - En phase d’exploitation, convertit l’état courant en tenseur PyTorch et ajoute une dimension batch.
     - Passe l’état au policy network pour calculer les Q-values associées à chaque action.
     - Sélectionne l’action ayant la Q-value maximale.
     - Retourne l’action choisie sous forme d’un entier utilisable par l’environnement.

Implémentation de l'apprentissage

7. Optimisation du modèle (DQN)

```python
def optimize_model():
    if len(memory) < batch_size:
        return
    # Échantillonnage aléatoire
    batch = random.sample(memory, batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)
    states = torch.FloatTensor(states)
    actions = torch.LongTensor(actions).unsqueeze(1)
    rewards = torch.FloatTensor(rewards)
    next_states = torch.FloatTensor(next_states)
    dones = torch.FloatTensor(dones)
    # Q(s,a) courant
    q_values = policy_net(states).gather(1, actions).squeeze()
    # Q cible : r + γ max Q_target(s',a')
    with torch.no_grad():
        max_next_q = target_net(next_states).max(1)[0]
        target_q = rewards + gamma * max_next_q * (1 - dones)
    # Fonction de perte MSE
    loss = nn.MSELoss()(q_values, target_q)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Implémentation de l'apprentissage

7. Optimisation du modèle (DQN)
   - Explication
     - Définit une fonction chargée d’optimiser le réseau principal (policy network) à partir de la mémoire de rejouement.
     - Vérifie que la mémoire contient au moins batch_size expériences avant de lancer l’apprentissage.
     - Sélectionne aléatoirement un mini-lot de transitions depuis la mémoire (experience replay).
     - Sépare les transitions en états, actions, récompenses, états suivants et indicateurs de fin d’épisode.
     - Convertit ces données en tenseurs PyTorch adaptés au calcul.
     - Calcule les Q-values courantes Q(s,a) prédites par le réseau principal pour les actions réellement prises.

Implémentation de l'apprentissage

7. Optimisation du modèle (DQN)
   - Explication
     - Calcule les Q-values cibles selon l’équation de Bellman, en utilisant le réseau cible et en désactivant le calcul du gradient.
     - Applique la fonction de perte MSE pour mesurer l’écart entre Q-values prédites et cibles.
     - Réinitialise les gradients, propage l’erreur en arrière (backpropagation) et met à jour les poids via l’optimiseur.

Implémentation de l'apprentissage

8. L’apprentissage

```
rewards_per_episode = []
steps_done = 0
for episode in range(episodes):
    state, _ = env.reset()
    state = one_hot(state, n_states)
    total_reward = 0
    done = False
    while not done:
        # Choix de l’action
        action = select_action(state, epsilon)
        # Interaction avec l’environnement
        next_state, reward, done, truncated, _ = env.step(action)
        next_state = one_hot(next_state, n_states)
        # Stockage de la transition
        memory.append((state, action, reward, next_state, done))
        # Passage à l’état suivant
        state = next_state
        total_reward += reward
        # Mise à jour du réseau
        optimize_model()
        # Mise à jour périodique du target network
        if steps_done % target_update_freq == 0:
            target_net.load_state_dict(policy_net.state_dict())
        steps_done += 1
    # Décroissance de ε
    epsilon = max(epsilon_min, epsilon * epsilon_decay)
    rewards_per_episode.append(total_reward)
```

Implémentation de l'apprentissage

8. L’apprentissage
   - Explication
     - Initialise les structures de suivi de l’apprentissage, notamment la liste des récompenses par épisode et le compteur global de pas.
     - Lance la boucle principale d’entraînement sur le nombre total d’épisodes définis.
     - Réinitialise l’environnement au début de chaque épisode et encode l’état initial en one-hot.
     - Initialise la récompense cumulée et le critère de fin d’épisode.
     - À chaque pas de temps, sélectionne une action selon la stratégie ε-greedy.
     - Exécute l’action dans l’environnement et récupère l’état suivant, la récompense et le signal de terminaison.
     - Encode l’état suivant et stocke la transition complète dans la mémoire de rejouement.

Implémentation de l'apprentissage

8. L’apprentissage
   - Explication
     - Met à jour l’état courant et accumule la récompense totale de l’épisode.
     - Appelle la fonction d’optimisation pour entraîner le réseau principal à partir de l’expérience stockée.
     - Met à jour périodiquement les poids du réseau cible à partir du réseau principal selon la fréquence définie.
     - Incrémente le compteur de pas global.
     - Applique la décroissance de epsilon afin de réduire progressivement l’exploration.
     - Enregistre la récompense totale obtenue pour chaque épisode afin d’analyser les performances d’apprentissage.

Implémentation de l'apprentissage

8. L’apprentissage
   - Explication
     - Met à jour l’état courant et accumule la récompense totale de l’épisode.
     - Appelle la fonction d’optimisation pour entraîner le réseau principal à partir de l’expérience stockée.
     - Met à jour périodiquement les poids du réseau cible à partir du réseau principal selon la fréquence définie.
     - Incrémente le compteur de pas global.
     - Applique la décroissance de epsilon afin de réduire progressivement l’exploration.
     - Enregistre la récompense totale obtenue pour chaque épisode afin d’analyser les performances d’apprentissage.

conclusion